

On the Concrete Hardness of LWR with a Power of Two Modulus

Jules Baudrin^{1*}, Rachelle Heim Boissier^{2*}, François-Xavier Standaert³

¹ Université Paris-Saclay, UVSQ, CNRS, Laboratoire de
mathématiques de Versailles, 78000, Versailles, France.

² Université Libre de Bruxelles, Belgium

³ Crypto Group, ICTEAM Institute, UCLouvain, Louvain-la-Neuve, Belgium

Abstract. LWR has been introduced by Banerjee et al. in 2012 as a deterministic variant of LWE. Since then, it has found many applications in the design of symmetric primitives and post-quantum schemes. Despite its deterministic nature, LWR is usually analyzed as LWE, under the (implicit) assumption that no improved attack can take advantage of the additional structure it provides. In this paper, we tackle this assumption in the context of power-of-two moduli and investigate the security of LWR against algebraic attacks in depth. For this purpose, we model its samples as the outputs of a vectorial Boolean function. We first observe that there are corner cases where the state-of-the-art linearisation attack of Arora and Ge does not apply. In contrast, we propose the first LWR-specific cryptanalysis, which applies in any parameter regime as long as the modulus is a power of two. We combine analyses of standard criteria such as the algebraic degree and the number of monomials in the secret with an analysis of the algebraic normal form, that we are able to express exactly in a compact representation by leveraging group action theory. Our results exhibit specificities in the structure of LWR that we are able to exploit. They allow refining and strengthening the understanding of this important problem, and systematically improve over the attack of Arora and Ge. They also put forward new tools for (symmetric) cryptanalysis, which we believe can be of independent interest.

1 Introduction

State of the art. Hard learning problems are of central importance in post-quantum cryptography. For example, the Learning With Errors (LWE) problem [35], and its variants Ring-LWE [33] or Module-LWE [16,30], are used as hardness assumptions in many encryption and signature schemes, including the new standards CRYSTALS-Kyber [37] and CRYSTALS-Dilithium [32].

One potential drawback of LWE is that it generates non-deterministic “noisy” samples. This for example limits its suitability in the design of deterministic (symmetric) primitives. In order to circumvent this problem, Banerjee et al. proposed a deterministic version of LWE, called Learning With Rounding (LWR),

* Work performed in part as post-doctoral researcher at UCLouvain.

where the noise addition is replaced by a deterministic rounding function [9,5]. LWR is known to be asymptotically as hard as LWE and has found applicability in the design of symmetric cryptographic primitives for different applications [11,8,23,2,19,25,42]. It is also the basis of (asymmetric) post-quantum cryptographic algorithms such as the SABER encryption scheme [21].

Despite LWE’s solid theoretical foundations [17], the selection of security parameters for lattice-based cryptosystems is for now mostly driven by heuristic estimates, based on the best-known attacks [3]. In this context, a usual assumption (e.g. found in the SABER specifications) is that “*the hardness of (ring or module) LWR can be analyzed as an LWE problem, since there is no known attacks that make use of the additional structure offered by these variants*”. Such an assumption is of course questionable given that the aforementioned reductions between LWE and LWR are not tight for the parameters typically considered in practice. The lack of a deeper understanding of the differences and similarities between LWE and LWR, and therefore the lack of LWR-specific cryptanalysis, is actually mentioned as one of the reasons why the NIST favored an LWE-based encryption scheme like Kyber over LWR-based ones like SABER [1].

In this paper, we question the assumption that attacks cannot use the structure offered by certain variants of learning problems, so that their application to LWE and LWR is identical. In particular, we propose the *first LWR-specific (algebraic) attack* and focus on the case of a power-of-two moduli 2^q , which has been considered in the design of PRFs [9,5], possibly specialized for distributed key management [23,25] or leakage applications [23]. Such PRFs are typically instantiated with a large noise level so that they can withstand adversaries obtaining large numbers of samples. Besides, LWR with power-of-two moduli has also been considered with more aggressive parameters, e.g. in constructions enforcing that the noise accumulates [19], where the number of samples is limited [21] or where additional secrets are used in the primitive [42].

Our contribution. As a first result, we show in Section 2 that the main algebraic approach to the cryptanalysis of LWE and LWR, namely the 2011 attack of Arora and Ge [6], and extensions thereof [3,39,38], does not systematically work when the modulus is a power of two. In particular, we exhibit corner case parameter regimes such that this attack is displayed as the best attack by lattice estimators whilst it is not applicable. In the same section, we thus introduce an LWR-specific approach which allows one to mount an algebraic attack *regardless of the parameter regime*. By identifying $\mathbb{Z}_{2^q}$ and $\mathbb{F}_2^q$, we observe that LWR samples are produced by a deterministic *vectorial Boolean function* mapping $2nq$ -bit words to p -bit words. Indeed, the LWR function maps $(\mathbf{a}, \mathbf{x}) \in \mathbb{Z}_{2^q}^n \times \mathbb{Z}_{2^q}^n$ (viewed as $\mathbb{F}_2^{qn} \times \mathbb{F}_2^{qn}$) to $\lfloor \langle \mathbf{a}, \mathbf{x} \rangle \rfloor_{2^p}$, where $\langle \mathbf{a}, \mathbf{x} \rangle = \sum_{i=0}^{n-1} a_i \times x_i$ denotes the dot product over $\mathbb{Z}_{2^q}$ and where the rounding function $\lfloor \cdot \rfloor_{2^p}$ removes the $q - p$ least significant bits. The coordinates of this function allow us to build Boolean polynomial equations in the secret. For example, the least significant coordinate function maps $(\mathbf{a}, \mathbf{x})$ to the LSB of the sample $\lfloor \langle \mathbf{a}, \mathbf{x} \rangle \rfloor_{2^p}$, *i.e.* the bit of index $q - p$ of $\langle \mathbf{a}, \mathbf{x} \rangle$. The rest of the paper then investigates the extent to which this approach,

besides working generically for any power-of-two modulus, can be improved in order to be *systematically more efficient* than the Arora-Ge approach.

We begin by showing in Section 3 that, when using equations derived from the LSB of the sample, the Boolean systems obtained have at most the same number of monomials as the systems of Arora and Ge. This is a somewhat surprising result given the number of variables and degree, which we also derive in the same section. It allows us to demonstrate that our new approach does not come with an additional cost compared to Arora and Ge’s, and in fact comes with a direct improvement by taking advantage of the practical benefits of working over $\mathbb{F}_2$.

Next, we tackle in Sections 4 and 5 a necessary step for the application of most known-plaintext attack paths relying on Boolean functions¹ and in particular algebraic attacks: the computation of the *Algebraic Normal Form* (ANF), *i.e.*, the polynomial representation over $\mathbb{F}_2$. Whilst our (tight) results on the number of monomials suggest that computing - let alone storing - the ANF is out-of-reach, we leverage group action theory to show that it is in fact possible using a relevant *system of representatives*. This allows us to compute and store a compact representation of the ANF of the least significant bit of LWR samples for 2^{q-p} up to 16. Interestingly, the size of the system of representatives depends *only* on the index of the bit considered and, in particular, not on n . More generally, we show that understanding the ANFs for $n = 2^{q-p}$ allows us to fully understand the ANF for *arbitrary large* n values, thanks to scaling theorems.

Finally, we show in Section 6 that we can identify rank defects and sparsity phenomena, and leverage them in order to improve algebraic attacks. We show that our *system of representatives* together with group action theory allow computing metrics that are relevant for linearisation attacks. More precisely, for 2^{q-p} up to 2^3 and for arbitrary n , we identify an improved linearisation family by grouping monomials that appear or vanish together for any $\mathbf{a}$, yielding a direct and systematic improvement of linearisation attacks. Furthermore, for 2^{q-p} up to 2^4 and for arbitrary n , we are able to compute the average number of non-zero terms per equation, allowing to precisely evaluate whether sparse linear algebra techniques allow additional improvements. We summarize our findings together with conclusive remarks and open questions in Section 7.

All in all, we systematically improve the linearisation attack of Arora and Ge, with the improvement varying depending on the exact parameters considered. Hence, our results are also the best attack in any parameters’ regime where Arora-Ge was so far considered the best cryptanalysis method (including regimes where it is, in fact, inapplicable). To the best of our knowledge, these improvements do not impact existing schemes which either use large noise levels or limit the adversaries in other ways (e.g., by limiting the number of samples in SABER). Yet, we believe they can contribute to the general understanding of LWR and help refining the security estimates of parameter regimes that could be considered in future use cases. For example, using the linear

¹ Attacks on LWR must be known-plaintext as $\mathbf{a}$ is drawn uniformly at random.

algebra constant $\omega = 3$, and for the parameters $2^{q-p} = 8$, $n = 128$ (resp. 256), we improve upon Arora and Ge’s attack by 15 bits, obtaining a concrete complexity of $2^{110.8}$ (resp. $2^{134.7}$) binary operations. More examples (including results with a conservative linear algebra constant $\omega = 2$ as used in the lattice estimator of Albrecht et al. [3]) and comparisons with the Arora-Ge attack are discussed in Section 7.1, and our code is available at the link <https://anonymous.4open.science/r/boolean-lwr/>.

Finally, besides offering a complete characterization of an important attack vector and exhibiting exploitable specificities in the structure of LWR, we also introduce tools for the exact modeling of algebraic attacks that may be of independent interest for cryptanalysis. We hope their application to the understanding of the Boolean properties of the dot product can be applied to other primitives. More generally, we also hope group actions which, to the best of our knowledge, have never been used before to study ANFs, will also find cryptanalytic applications beyond the LWR case in the future.

General notations. In the following, the finite field with two elements is denoted by $\mathbb{F}_2 = \{0, 1\}$, for any $n \in \mathbb{N}$, $\llbracket 0, n \rrbracket$ is defined by $\llbracket 0, n \rrbracket := \{0, 1, \dots, n\}$ and for any $q \in \mathbb{N}$, the quotient ring $\mathbb{Z}/2^q\mathbb{Z}$ is denoted by $\mathbb{Z}_{2^q}$. The Hamming weight of a binary vector $v \in \mathbb{F}_2^s$ is denoted by $hw(v)$, the support of v is denoted by $\text{Supp}(v)$, and the dot product of two n -long vectors is denoted by $\langle \cdot, \cdot \rangle_n$ or $\langle \cdot, \cdot \rangle$ if n is clear from context.

2 Algebraic attacks with power-of-two moduli

Whilst the algebraic attack of Arora and Ge is the best attack for ranges of parameters with low noise and large numbers of samples according to lattice estimators, we show in this section that the applicability of this technique is non-trivial when considering power-of-two moduli. We begin this section by briefly recalling the definitions of the LWE and LWR problems in their power-of-two moduli versions. We let $n, q \in \mathbb{N}^*$ be positive integers, $\mathbf{k} \in \mathbb{Z}_{2^q}^n$ be a secret drawn uniformly at random, and χ be a Gaussian distribution over $\mathbb{Z}_{2^q}$.

Learning With Errors [35]. Learning With Errors (LWE) formalizes the problem of solving a noisy linear system. In its search version, it consists in finding $\mathbf{k} \in \mathbb{Z}_{2^q}^n$ given samples from the distribution

$$\mathcal{D}^{\text{LWE}} = \{ (\mathbf{a}, \langle \mathbf{a}, \mathbf{k} \rangle + e), \mathbf{a} \xleftarrow{\$} \mathbb{Z}_{2^q}^n, e \leftarrow \chi \}.$$

Decision LWE is the problem of distinguishing $\mathcal{D}^{\text{LWE}}$ from the distribution $\mathcal{D}_0 = \{ (\mathbf{a}, r) \mid \mathbf{a} \xleftarrow{\$} \mathbb{Z}_{2^q}^n, r \xleftarrow{\$} \mathbb{Z}_{2^q} \}$.

Learning With Rounding [9]. Learning With Rounding (LWR) can be viewed as an LWE variant in which the noise is replaced by a deterministic rounding. In the power-of-two modulus case, letting $p \in \llbracket 1, q-1 \rrbracket$, the rounding function

$\lfloor \cdot \rfloor_{2^p} : \mathbb{Z}_{2^q} \rightarrow \mathbb{Z}_{2^p}$ removes the $q - p$ least significant bits. In its search version, LWR consists in finding $\mathbf{k}$ given samples from the distribution

$$\mathcal{D}^{\text{LWR}} = \{ (\mathbf{a}, \lfloor \langle \mathbf{a}, \mathbf{k} \rangle \rfloor_{2^p}), \mathbf{a} \xleftarrow{\$} \mathbb{Z}_{2^q}^n \}.$$

Similarly to Decision LWE, Decision LWR aims at distinguishing $\mathcal{D}^{\text{LWR}}$ from $\mathcal{D}_1 = \{(\mathbf{a}, r) \mid \mathbf{a} \xleftarrow{\$} \mathbb{Z}_{2^q}^n, r \xleftarrow{\$} \mathbb{Z}_{2^p}\}$.²

2.1 The Arora-Ge linearisation technique

We provide a brief description of the algebraic attack on LWE by Arora and Ge [6]. Assume that the noise is drawn in a finite set E of cardinality $|E|$. In this case, for any LWE sample $(\mathbf{a}, s)$ where $s := \langle \mathbf{a}, \mathbf{k} \rangle + \tilde{e}$, the secret $\mathbf{k}$ is a root of the following equation:

$$\prod_{e \in E} (s - \langle \mathbf{a}, \mathbf{x} \rangle - e) = 0,$$

where $\langle \mathbf{a}, \mathbf{x} \rangle$ is the linear equation $\sum_{i \in \llbracket 0, n-1 \rrbracket} a_i x_i$ in the unknowns $x_0, \dots, x_{n-1}$. If the attacker receives M samples $\{(\mathbf{a}^{(i)}, s^{(i)})\}_{i \in \llbracket 1, M \rrbracket}$, she can recover the secret by solving the polynomial system:

$$\left\{ \prod_{e \in E} (s^{(i)} - \langle \mathbf{a}^{(i)}, \mathbf{x} \rangle - e) = 0 \right\}_{i \in \llbracket 1, M \rrbracket}.$$

As observed in the original paper of Arora and Ge, this system can be linearised when M is of the order of the number of monomials. This multivariate polynomial system has n unknowns $x_0, \dots, x_{n-1}$, and each equation has degree at most $|E|$. The system thus has at most $\binom{n+|E|}{|E|}$ monomials. Straightforward linearisation yields a time complexity of $\binom{n+|E|}{|E|}^\omega$, with ω the linear algebra constant (e.g. $\omega = 3$ using Gaussian elimination). Subsequent works have studied the complexity of solving this system using Gröbner bases, but their results apply only when the modulus is either odd [3] or prime [38] - in particular, not a power-of-two.³

In the LWE case, the noise is Gaussian. The aforementioned works [6, 3, 39, 38] compute the trade-off between a number of samples large enough so that the system can be solved, and low enough so that the noise belongs to a set E of smaller cardinality with high probability. We do not go into the details here as this does not impact our analysis.

² The only difference with $\mathcal{D}_0$ is that r is drawn in $\mathbb{Z}_{2^p}$ rather than $\mathbb{Z}_{2^q}$.

³ In a nutshell, these results show that the data complexity, which is of the same order as the number of monomials in the original attack, can be improved at the cost of a higher time complexity.

The case of LWR. In the case of LWR, the Arora-Ge attack can be adapted by observing that any LWR sample $(\mathbf{a}, s := \lfloor \langle \mathbf{a}, \mathbf{k} \rangle \rfloor_{2^p})$ provides an equation in the secret $\prod_{e \in \mathbb{Z}_{2^{q-p}}} (s || e - \langle \mathbf{a}, \mathbf{x} \rangle)$ where $s || e$ is the bit concatenation of the p -bit sample s and the $q - p$ -bit “error value” e .

2.2 On the applicability of the Arora-Ge technique

As previously mentioned, the lattice estimator of Albrecht et al. [4] cites the attack of Arora and Ge as the best attack for some parameters with low noise, including with a power-of-two modulus (e.g. similar to the parameters of SABER $n = 512$, $q = 2^{13}$, $p = 2^{10}$ but without the existing bound on the number of samples). Yet, we argue in this section that in this case, the Arora-Ge attack is not systematically applicable.

An important observation. Essentially, problems arise whenever the set E is a somewhat large interval. For simplicity, we assume that the cardinality of E is a power of two $|E| := 2^c$, but it is straightforward to generalise our observation to an arbitrary cardinality. A trivial yet significant observation is the following. For any $y \in \mathbb{Z}_{2^q}$, $\prod_{e \in E} (y - e)$ is the product of 2^c consecutive elements. Among those elements, 2^{c-1} are even, 2^{c-2} are dividable by 4, and so on. All in all, for any $y \in \mathbb{Z}_{2^q}$, $\prod_{e \in E} (y - e)$ is dividable by

$$\prod_{i=1}^c 2^{2^{c-i}} = 2^{2^c - 1}.$$

In particular, the function $y \mapsto \prod_{e \in E} (y - e) \bmod 2^{2^c - 1}$ is the zero function. Thus, whenever $2^{2^c - 1} \geq 2^q$ the Arora-Ge equations provides *no information on the secret*. In the case of LWR, using the exact same reasoning, the Arora-Ge equations are the trivial equation $0 = 0$ whenever $2^{q-p} - 1 \geq q$. For example, our observation implies that the technique of Arora and Ge is not applicable to the distributed PRF LaKey [25] which has parameters $q - p = 4$ and $q = 12$ so that $2^{q-p} - 1 = 15 > q = 12$.

Lattice estimator errors. In the lattice estimator by Albrecht et al. [4], Arora-Ge is given as the best attack against LWE with parameters

$$(2^q, 2^p) \in \{(2^7, 2^4), (2^6, 2^3), (2^5, 2^2)\}.$$

and $n = 512$ (and in fact, most reasonably large n values). This can be verified by entering the following code with $(q1, p1) \in \{(7, 4), (6, 3), (5, 2)\}$ in the estimator:

```
params = LWE.Parameters(n=300, q=2**q1,
    Xs=ND.Uniform(-2**q1-1,0), Xe=ND.Uniform(-2**(q1-p1)-1,0))
LWE.estimate(params)
```

Yet, in these cases, $q - p = 3$ and thus $2^{q-p} - 1 = 7 \geq q$ – the attack does not apply.

Generalizations of the observation. The previous observation stems from the fact that $\mathbb{Z}_{2^q}$ is not an integral domain (unless $q = 1$) and can be generalized to other rings $\mathbb{Z}_s$. In particular, when s admits small prime divisors, such corner cases will occur frequently. As an example if $s = 6$ and if E is made of more than 3 consecutive values, then the Arora-Ge equation will provide no information since a product of three consecutive integers is necessarily dividable by 6.

2.3 On the complexity of the Arora-Ge technique

The previous observation implies that in several cases, the Arora-Ge attack simply cannot be used in the case of power-of-two moduli. However, for real-life parameters, the error set E is typically sufficiently small compared to the size of q . For example, in SABER, $q - p = 3$ and $q = 13$ and thus $2^{q-p} - 1 \ll q$. Nevertheless, our observation impacts the concrete complexity of solving the linearised system.

Solving a (homogeneous) linear system over a (finite) ring is not as straightforward as solving it over a field - classical techniques such as Gaussian elimination must be tailored to make up for the fact that not all elements are invertible. In the case of $\mathbb{Z}_{2^q}$, two approaches can be considered. The first one is to solve the system directly in the ring, which costs upwards of $q \cdot n^\omega$ binary operations for a square matrix of size n . In this case, q is of course a generous lower bound on the constant since any arithmetic operation in $\mathbb{Z}_{2^q}$ beyond addition costs strictly more than q binary operations [36,12]. The second one, *Hensel lifting*, allows in general to obtain a constant equal to q . It simply consists in solving the system modulo 2, then lifting the obtained solution(s) to find solution(s) modulo 4, and so on by only solving a Boolean linear system at each step. Details can be found in [24, Section 15.4]. However, our former observation shows that it cannot be directly applied to the equations obtained with Arora-Ge. Indeed, reducing modulo 2^t for any $t \leq 2^c - 1$ yields all solutions in the ring: in this case, one is again solving the trivial equation $0 = 0$. Thus, the smallest ring in which the equations can be considered is $\mathbb{Z}_{2^{2^c}}$. Solving the system thus costs upwards of $n^\omega \cdot 2^c$ binary operations for a square matrix of size n , with $c = q - p$ in the case of LWR.

2.4 A new Boolean approach

In this paper, we propose a natural yet new approach to the cryptanalysis of LWR. Instead of looking at the LWR function $(\mathbf{a}, \mathbf{x}) \mapsto \lfloor \langle \mathbf{a}, \mathbf{x} \rangle \rfloor_{2^p}$ as a function from $\mathbb{Z}_{2^q}^n \times \mathbb{Z}_{2^q}^n$ to $\mathbb{Z}_{2^p}$, we can consider this function as mapping two vectors of nq bits to a p -bit vector. By identifying $\mathbb{Z}_{2^q}$ and $\mathbb{F}_2^q$ (*i.e.* identifying integers of $\llbracket 0, 2^q - 1 \rrbracket$ with their binary decompositions), the LWR function can be seen as the following *vectorial Boolean function*:

$$\begin{aligned} (\mathbb{F}_2^q)^n \times (\mathbb{F}_2^q)^n &\longrightarrow \mathbb{F}_2^p \\ (\mathbf{a}, \mathbf{x}) &\longmapsto \lfloor \langle \mathbf{a}, \mathbf{x} \rangle \rfloor_{2^p}. \end{aligned}$$

In particular, a function that maps two nq -bit vectors $(\mathbf{a}, \mathbf{x})$ to any (product of) bit(s) of $[\langle \mathbf{a}, \mathbf{x} \rangle]_{2^v}$ is a Boolean function. We will show in the sequel that this approach allows us to generate Boolean polynomial equations which, contrarily to the Arora-Ge's equation, provide information on the secret *regardless of the parameter regime*. Furthermore, we manage to exploit them in a way that is systematically cheaper.

In the following, the coordinates of $\mathbf{x}$ are indicated by a single subscript: $\mathbf{x} = (x_0, \dots, x_{n-1}) \in \mathbb{Z}_{2^q}^n$. Each $x_i \in \mathbb{Z}_{2^q}$ is a q -bit vector. We refer to the bit of index j of x_i by using a second subscript, $x_{i,j} \in \mathbb{F}_2$, and reserve bold characters, e.g. $\mathbf{x}, \mathbf{a}, \mathbf{u}, \mathbf{v}$ for vectors of $\mathbb{Z}_{2^q}^n$.

Algebraic Normal Form. It is well-known that any Boolean function $f: \mathbb{F}_2^s \rightarrow \mathbb{F}_2$ is a multivariate polynomial function in s variables $X_0, \dots, X_{s-1}$. This polynomial representation is unique modulo the field equations $X_0^2 - X_0, \dots, X_{s-1}^2 - X_{s-1}$ and known as the *Algebraic Normal Form* (ANF) of f . In the following, any multivariate polynomial will thus *by default* be considered modulo $X_i^2 - X_i$ for all i . Since the LWR function is a vectorial Boolean function, each coordinate or product of coordinates admits an ANF. In particular, it is possible to mount algebraic attacks on LWR by solving a Boolean polynomial system, as long as one can compute and store it.

Generically, for any Boolean function $F: \mathbb{Z}_{2^q}^n \times \mathbb{Z}_{2^q}^n \rightarrow \mathbb{F}_2$, the ANF of F is expressed as

$$F: (\mathbf{a}, \mathbf{x}) \mapsto \sum_{\mathbf{u}, \mathbf{v} \in \mathbb{Z}_{2^q}^n} \alpha_{\mathbf{u}, \mathbf{v}}(F) \mathbf{a}^{\mathbf{u}} \mathbf{x}^{\mathbf{v}}, \text{ where}$$

$$\alpha_{\mathbf{u}, \mathbf{v}}(F) \in \mathbb{F}_2 \quad \text{and} \quad \mathbf{a}^{\mathbf{u}} \mathbf{x}^{\mathbf{v}} = \prod_{i=0}^{n-1} a_i^{u_i} x_i^{v_i} = \prod_{i=0}^{n-1} \prod_{j=0}^{q-1} a_{i,j}^{u_{i,j}} x_{i,j}^{v_{i,j}}.$$

Since we mostly consider Boolean functions, we emphasize that the exponential notation $x_i^{u_i}$ for integers x_i and u_i *does not* denote the product of x_i with itself u_i times, but the Boolean monomial $\prod_{u_i \neq 0} x_i$. In the sequel, we will use $\times$ to denote a (modular) integer product.

Example 1. Consider variables $\mathbf{a} = (a_0, a_1)$ and $\mathbf{x} = (x_0, x_1)$ in $\mathbb{Z}_{2^q}^2$ (each a_i, x_i is a q -bit variable). Our notations yield

$$\mathbf{a}^{(1,2)} \mathbf{x}^{(0,2)} = a_0^1 a_1^2 x_1^2 = a_{0,0} a_{1,1} x_{1,1}. \quad \triangleright$$

In contexts where we need to refer to the transformation $y \mapsto y^m$ as a *function*, we introduce the slightly redundant notation π_m . This is particularly useful for composition. By definition, when m is of Hamming weight 1, *i.e.* $m = 2^j$, π_{2^j} is nothing more than the projection on the coordinate of index j . For example, $\pi_{2^j} \circ \langle \mathbf{a}, \mathbf{x} \rangle$ returns the bit of index j of the dot product between $\mathbf{a}$ and $\mathbf{x}$. Understanding this function's ANF can be viewed as understanding algebraically the recursive propagation of the carries through the modular additions and multiplications involved in a dot product. When $hw(m) > 1$, π_m computes a product of bits (whose ANF will mostly consist of an intricate product of carries).

Important notation. For shorter notation, $F^{m,n}$ stands in the following for the Boolean function $\mathbb{Z}_{2^q}^n \times \mathbb{Z}_{2^q}^n \rightarrow \mathbb{F}_2$ $(\mathbf{a}, \mathbf{x}) \mapsto \pi_m \circ \langle \mathbf{a}, \mathbf{x} \rangle_n$.

Example 2. Since the rounding removes the $q-p$ least significant bits, the function $F^{2^{q-p},n} = \pi_{2^{q-p}} \circ \langle \cdot, \cdot \rangle$ maps $(\mathbf{a}, \mathbf{x})$ to the LSB of the *sample* $\lfloor \langle \mathbf{a}, \mathbf{x} \rangle \rfloor_{2^p}$. $\triangleright$

Known-plaintext setting. Since the LWR function is evaluated on random inputs, LWR should be studied in the known-plaintext setting. When F is a function of public variables $\mathbf{a}$ and private variables $\mathbf{x}$, it is convenient to view it as a polynomial in $\mathbf{x}$ with coefficients in $\mathbb{F}_2[\mathbf{a}]$, i.e. as:

$$F = \sum_{\mathbf{v} \in \mathbb{Z}_{2^q}^n} \alpha_{\mathbf{v}}(F) \mathbf{x}^{\mathbf{v}}, \text{ where } \alpha_{\mathbf{v}}(F) := \sum_{\mathbf{u} \in \mathbb{Z}_{2^q}^n} \alpha_{\mathbf{u},\mathbf{v}}(F) \mathbf{a}^{\mathbf{u}} \in \mathbb{F}_2[\mathbf{a}]. \quad (1)$$

This decomposition can in fact be transformed by an adversary into an equation. Indeed, if one has access to both the ANF of F and the value of a sample $\mathbf{a} \in (\mathbb{Z}_{2^q})^n$, then each evaluation $\alpha_{\mathbf{v}}(F)(\mathbf{a}) \in \mathbb{F}_2$ can be computed and one thus obtains a Boolean equation in the unknowns $\mathbf{x}$.

We therefore distinguish between the set of exponents in $\mathbf{a}$ and $\mathbf{x}$ on the one hand, and the exponents in $\mathbf{x}$ on the other, that we respectively define by:

$$\text{Exp}(F) = \{(\mathbf{u}, \mathbf{v}) \in (\mathbb{N}^n)^2, \alpha_{\mathbf{u},\mathbf{v}}(F) \neq 0\}, \quad \text{Exp}_{\mathbf{x}}(F) = \{\mathbf{v} \in \mathbb{N}^n, \alpha_{\mathbf{v}}(F) \neq 0\}.$$

In the previous definition, we let $\mathbf{v}$ (resp. $\mathbf{u}$) be a vector of positive integers, and not of integers modulo 2^q . We will often make use of this notation which is unambiguous : indeed, if $u_i \geq 2^q$ for some i then for any $\mathbf{x} \in \mathbb{Z}_{2^q}$, $\mathbf{x}^{\mathbf{v}} = 0$ since there exists a bit of (high) index j such that $u_{i,j} = 1$ and $x_{i,j} = 0$. We also define the *algebraic degree* of F in $\mathbf{x}$ and the coefficients in $\mathbf{x}$ of F by:

$$\deg_{\mathbf{x}}(F) := \max_{\mathbf{v} | \alpha_{\mathbf{v}}(F) \neq 0} \text{hw}(\mathbf{v}) \quad \text{and} \quad \text{Coefficients}_{\mathbf{x}}(F) = \{\alpha_{\mathbf{v}}(F), \mathbf{v} \in \text{Exp}_{\mathbf{x}}(F)\}.$$

Finally, for any fixed value $\mathbf{a} \in \mathbb{Z}_{2^q}^n$, we define the specialized function $F_{\mathbf{a}}$ by:

$$F_{\mathbf{a}}: \mathbb{Z}_{2^q}^n \rightarrow \mathbb{F}_2 \quad \mathbf{x} \mapsto F(\mathbf{x}, \mathbf{a}).$$

3 A new attack path against LWR

3.1 Challenges and objectives of our approach

Our goal is to explore the Boolean modeling of LWR. Overall, switching from non-linear systems over $\mathbb{Z}_{2^q}$ to systems over $\mathbb{F}_2$ might seem like a bad idea. Indeed, independently of the technique that is used, the complexity of solving a non-linear system is mainly driven by the degree and the number of monomials. Without further investigation, this therefore implies switching from equations in $\mathbb{Z}_{2^q}$ with at most $\binom{n+2^{q-p}}{2^{q-p}}$ monomials in $\mathbb{Z}_{2^q}$ (see Section 2.1) to Boolean equations which could have up to 2^{nq} monomials in $\mathbf{x}$, or up to $\sum_{i=0}^d \binom{nq}{i}$ monomials if the degree of the equations was known to be bounded by d .

The main result of this section is that our Boolean point of view *does not* come with an increased number of monomials nor a higher complexity. On the contrary, we prove that the number of monomials in the ANF of the bit of index 2^{q-p} of $(\mathbf{a}, \mathbf{x}) \mapsto \langle \mathbf{a}, \mathbf{x} \rangle$, that is, the least significant bit available to an adversary, admits at most $\binom{n+2^{q-p}}{2^{q-p}}$ Boolean monomials, *i.e.* the exact same number of monomials as $\mathbb{Z}_{2^q}$ -equations of degree at most 2^{q-p} in n variables. The least significant bits of the secret can thus be recovered in about $\binom{n+2^{q-p}}{2^{q-p}}^\omega$ binary operations, that is, for a cost at least 2^{q-p} times lower than the cost of solving Arora-Ge equations as discussed in Section 2.3. We further show in Section 3.3 that the rest of the secret can also be recovered for a negligible additional cost.

However, this improvement is obtained *only if* the ANF of each coefficient in $\mathbf{x}$, *i.e.* $\alpha_v(F^{m,n})$, is known. These ANFs are indeed necessary to derive an equation in $\mathbf{x}$ from each sample $(\mathbf{a}, s_{\mathbf{a}})$. The coefficients $\alpha_v(F^{m,n})$ are *a priori* random-looking polynomials of $\mathbb{F}_2[\mathbf{a}]$ and it is rare to have information beyond *e.g.* a bound on the algebraic degree of symmetric primitives [14,27]. As detailed in Section 3.4, both the time and memory complexities to compute the ANF are typically exceptionally high. The goal of Section 5 will be to prove that this usually prohibitive step can be efficiently achieved in the case of the LWR function, by taking advantage of symmetries described in Section 4.

For now, we prove in Section 3.2 the announced bound on the number of monomials in the Boolean coordinates of the LWR function. We then discuss in Section 3.3 the final step to mount a full key-recovery for a marginal additional cost (under the assumption that the ANF can be computed).

3.2 Number of terms of the Boolean coordinates of the dot product

Recall that $F^{m,n}$ stands for the Boolean function $\mathbb{Z}_{2^q}^n \times \mathbb{Z}_{2^q}^n \rightarrow \mathbb{F}_2$ $(\mathbf{a}, \mathbf{x}) \mapsto \pi_m \circ \langle \mathbf{a}, \mathbf{x} \rangle_n$.

ANF of modular addition. To study the ANF of (product of) bits of the dot product, we rely on the ANF of modular addition, which has been previously investigated by Braeken and Semaev [15]. We denote by SUM^n the modular addition of n elements, namely: $\text{SUM}^n : \mathbb{Z}_{2^q}^n \rightarrow \mathbb{Z}_{2^q}$ $\mathbf{x} \mapsto \sum_{i=0}^{n-1} x_i$. Equivalently, $\text{SUM}^n(\mathbf{x}) = \langle \mathbf{1}, \mathbf{x} \rangle_n$, where $\mathbf{1}$ is the all-1 vector of length n . For any $n, m \in \mathbb{N}$, the set of n -long vectors that sum to m is denoted by $S_m^n := \left\{ \mathbf{c} \in \mathbb{N}^n \mid \sum_{i=0}^{n-1} c_i = m \right\}$.

Theorem 1 (ANF of modular addition [15, Theorem 1]). *For any $m \in \mathbb{N}$, and any $x, y \in \mathbb{Z}_{2^q}$, it holds that: $(x + y)^m = \sum_{c=0}^m x^{m-c} y^c$. More generally for any $n \in \mathbb{N}^*$, it holds that:*

$$\pi_m \circ \text{SUM}^n(\mathbf{x}) = \sum_{\mathbf{c} \in S_m^n} \mathbf{x}^{\mathbf{c}}. \quad (2)$$

We refer to the original paper for the proof [15, Theorem 1] or to an alternative one based on monomial prediction [29, Theorem 2]. It should be noted

that the ANF of $\pi_m \circ \text{SUM}^n$ does not depend on the size q of the integers. Indeed, if $\mathbf{c} \in S_m^n$, then each c_i satisfies $c_i \leq m$ and is therefore of size less than $\lfloor \log_2(m) \rfloor + 1$. This implies that only bits of index lower than or equal to $\lfloor \log_2(m) \rfloor + 1$ are actually involved. This essentially captures the fact that the i -th carry depends on the $(i-1)$ least significant bits of each term – a natural consequence of the triangular nature of modular addition. Next, we derive the exact degree and the set of exponents of the functions $\pi_m \circ \text{SUM}^n$ from the previous theorem.

Proposition 1. *Let $n, m \in \mathbb{N}^*$. The following statements hold.*

- (i) **Exponents.** $\text{Exp}(\pi_m \circ \text{SUM}^n) = S_m^n$ and $|\text{Exp}(\pi_m \circ \text{SUM}^n)| = \binom{n+m-1}{m}$.
- (ii) **Degree.** $\deg(\pi_m \circ \text{SUM}^n) \leq m$. If $m \leq n$, then $\deg(\pi_m \circ \text{SUM}^n) = m$.

Proof. (i). This is an immediate rewriting of Theorem 1. The cardinality of S_m^n is equal to the number of monomials of degree m in n variables (in $R[x_1, \dots, x_n]$ with R an arbitrary ring), which is well-known (see e.g. [6, Section 3]) to be equal to $\binom{n+m-1}{m}$ (as a consequence of the “stars and bars” reasoning and Pascal’s triangle formula).

(ii). For any $\mathbf{c} \in S_m^n$, $\deg(\mathbf{x}^{\mathbf{c}}) = \sum_{i=0}^{n-1} hw(c_i) \leq \sum_{i=0}^{n-1} c_i = m$. When $m \leq n$, for any $\mathcal{I} \subset \{0, \dots, n-1\}$ of cardinality $|\mathcal{I}| = m$, the vector $\mathbf{c} = (c_0, \dots, c_{n-1})$ such that $c_i = 1$ if $i \in \mathcal{I}$ and $c_i = 0$ otherwise belongs to S_m^n and corresponds to the monomial $\mathbf{x}^{\mathbf{c}} = \prod_{i \in \mathcal{I}} x_i^{c_i} = \prod_{i \in \mathcal{I}} x_{i,0}$ which has degree m . $\square$

ANF of modular multiplication. We now derive similar bounds for $F^{m,n}$. To do so, we also rely on the ANF of modular multiplication as described by Braeken and Semaev [15].

Theorem 2 (ANF of modular multiplication [15, Theorem 2]). *For any $m \in \mathbb{N}^*$, and any $a, x \in \mathbb{Z}_{2^q}$, it holds that:*

$$(a \times x)^m = \sum_{\substack{\mathbf{e} \in S_m^q \\ \forall i \in \llbracket 0, q-1 \rrbracket, 2^i \mid e_i}} a^{g(\mathbf{e})} x^{h(\mathbf{e})},$$

with for any j , $\pi_{2^j} \circ g(\mathbf{e}) := \bigvee_{i=0}^{q-1} \pi_{2^i} \left(\frac{e_j}{2^j} \right)$ and $h(\mathbf{e}) := \bigvee_{i=0}^{q-1} \frac{e_i}{2^i}$.

As pointed out in [15, Remark 2], this formula is not reduced because the function $\mathbf{e} \mapsto (g(\mathbf{e}), h(\mathbf{e}))$ is not injective: some terms $a^u x^v$ may appear an even number of times and cancel out.

Example 3 (Extracted from [15, Example 2]). Assume $m = 5$. The ordered partitions $\mathbf{e}$ of 5 such that $2^i | e_i$ are $(5, 0, 0)$, $(1, 4, 0)$, $(1, 0, 4)$, $(3, 2, 0)$. Dividing e_i by 2^i , we obtain the vectors $(5, 0, 0)$, $(1, 2, 0)$, $(1, 0, 1)$, $(3, 1, 0)$. This yields:

$$(a_0 \times x_0)^5 = a_0^1 x_0^5 + a_0^3 x_0^3 + a_0^5 x_0^1 + a_0^3 x_0^3 = a_0^1 x_0^5 + a_0^5 x_0^1 = a_{0,0} x_{0,0} x_{0,2} + a_{0,0} a_{0,2} x_{0,0}.$$

$\triangleright$

Corollary 1. *For any $m \in \mathbb{N}^*$, it holds that $\text{Exp}_x((a, x) \mapsto (a \times x)^m) \subset \llbracket 1, m \rrbracket$.*

Proof. According to Theorem 2, it is sufficient to prove that for any $\mathbf{e} \in S_m^q$, $h(\mathbf{e}) \subset \llbracket 1, m \rrbracket$, i.e. $1 \leq h(\mathbf{e}) \leq m$. By definition, $\sum_{i=0}^{q-1} e_i = m > 1$, so $\mathbf{e} \neq (0, \dots, 0)$ and thus $h(\mathbf{e}) \neq 0$. Furthermore it holds that:

$$h(\mathbf{e}) = \bigvee_{i=0}^{q-1} \frac{e_i}{2^i} \leq \sum_{i=0}^{q-1} \frac{e_i}{2^i} \leq \sum_{i=0}^{q-1} e_i = m. \quad \square$$

As for modular addition, this corollary captures the ‘triangular nature’⁴ of the ANF of modular multiplication, easily observed in the classical shift-and-add multiplication algorithm in base 2. Indeed, the bit of index j of $a \times x$ only depends linearly on x_j , non-linearly on bits of index $i < j$ (because of the carries) and the constant term 1 does not appear (as $a \times 0 = 0$). This corresponds to the $1 + 2^j - 1 = 2^j$ terms of $(a \times x)^{2^j}$.

Finally, we obtain the announced bound on the number of terms in $\mathbf{x}$ in the ANF of (products of) bits of the dot product.

Theorem 3 (Exponents and degree of $F^{m,n}$). *For any $m, n \in \mathbb{N}^*$, the following statements hold.*

- (i) **Exponents.** $\text{Exp}_x(F^{m,n}) \subset \bigcup_{i=1}^m S_i^n$ and thus $|\text{Exp}_x(F^{m,n})| \leq \binom{n+m}{m} - 1$.
- (ii) **Degree.** $\deg(F^{m,n}) \leq m$. If $m \leq n$, then $\deg_x(F^{m,n}) = m$.
- (iii) **Variables.** $F^{m,n}$ depends on at most $n \cdot (\lfloor \log_2(m) \rfloor + 1)$ variables of $\mathbf{x}$.

Proof. By Theorem 1, $F^{m,n} = \sum_{\mathbf{c} \in S_m^n} \prod_{i=0}^{n-1} (a_i \times x_i)^{c_i}$. For any $\mathbf{c} \in S_m^n$ and any i , by Corollary 1, it holds that $\text{Exp}_{x_i}((a_i \times x_i)^{c_i}) \subset \llbracket 1, c_i \rrbracket$. Therefore, the monomials $\mathbf{x}^{\mathbf{u}}$ in the ANF of $F^{m,n}$ necessarily satisfy $\sum_{i=0}^{n-1} u_i \leq \sum_{i=0}^{n-1} c_i = m$ since $\mathbf{c} \in S_m^n$ and $\mathbf{u} \neq (0, \dots, 0)$. This directly implies (i), of which (iii) is a direct consequence since the binary decomposition of m is of size $\lfloor \log_2(m) \rfloor + 1$. The algebraic degree of such a monomial is equal to $\sum_{i=1}^n hw(u_i) \leq \sum_{i=1}^n u_i \leq m$, which implies (ii). It holds that $\deg_x(F^{m,n}) = \max_{\mathbf{a} \in \mathbb{Z}_{2^q}^n} \deg(F_{\mathbf{a}}^{m,n})$. As shown before, if $\mathbf{a} = \mathbf{1}$ then $F_{\mathbf{a}}^{m,n} = \langle \mathbf{1}, \mathbf{x} \rangle = \pi_m \circ \text{SUM}^n(\mathbf{x})$. By Proposition 1, when $m \leq n$, it then holds that $\deg(F_{\mathbf{a}}^{m,n}) = m$, which proves that $\deg_x(F^{m,n}) = m$ in that case. $\square$

Remark 1. Our experimental results suggest that the inclusion of Theorem 3, $\text{Exp}_x(F^{m,n}) \subset \bigcup_{i=1}^m S_i^n$, is in fact an equality whenever m is a power of two. This has been verified for m up to 16 and for arbitrary large n , thanks to the dedicated ANF analysis described in Sections 4 and 5. The proof of this fact is an interesting open question.

Since $\mathbf{a}$ and $\mathbf{x}$ play symmetric roles in the dot product, the results of Theorem 3 apply to the degree, set of exponents and number of variables “in $\mathbf{a}$ ”. As for modular addition and multiplication, Item (iii) above states that the ANF of $F^{m,n}$ (i.e., bits of the dot product) *does not* depend on the size q of the input integers. This is again due to the triangular nature of modular operations.

⁴ This triangularity property being also shared by modular addition, it also extends to any polynomial function from $\mathbb{Z}_{2^q}$ to itself, see e.g. [34].

3.3 A solving strategy based on Boolean equations

As announced, applying Theorem 3 to the least significant bit available to an adversary, *i.e.* the bit of index $q - p$ (or equivalently $m = 2^{q-p}$),⁵ indicates that the corresponding Boolean equations will have up to $\binom{n+2^{q-p}}{2^{q-p}}$ monomials that only depends in $q - p + 1$ least significant variables $x_{i,j}$ ($0 \leq j \leq q - p$) for each i . This implies that these bits can be recovered by a straightforward linearisation at the cost of about $\binom{n+2^{q-p}}{2^{q-p}}$ in data, $\binom{n+2^{q-p}}{2^{q-p}}^2$ in memory and $\binom{n+2^{q-p}}{2^{q-p}}^\omega$ in time.

While the entire secret is not yet recovered, the remaining bits of index $i \geq q - p + 1$ can be recovered for a negligible cost. Indeed, for any ℓ , $F^{2^\ell, n}(\mathbf{a}, \mathbf{x})$ can be expressed as $F^{2^\ell, n}(\mathbf{a}, \mathbf{x}) = \sum_{i=0}^{n-1} a_{i,\ell} x_{i,\ell} + c_\ell$ where the carry c_ℓ only depends on bits of index strictly lower than ℓ of $\mathbf{a}$ and $\mathbf{x}$. Since the lower bits of $\mathbf{x}$ are recovered and the ones of $\mathbf{a}$ are known, this carry value can be computed and this provides a Boolean linear equation in n unknowns, $x_{0,2^\ell}, \dots, x_{n-1,2^\ell}$. The most significant bits of the secret can thus be recovered one after the other. The extra cost corresponds to solving $p - 1$ *linear* systems in n unknowns over $\mathbb{F}_2$, which is negligible before the cost of the initial non-linear system solving, both in time and memory. The data cost is not increased since we leverage the so-far unused bits of the samples that were collected in the first step.

3.4 Towards the computation of the full ANF

The benefits of this new attack path being clearly established, it remains to gauge whether or not it can be efficiently implemented. In contrast with the Arora-Ge equations where for each sample, the coefficients in $\mathbf{a}$ can be precomputed (using multinomial expansions) and quickly evaluated to obtain an equation in the unknowns, this is clearly less trivial with our Boolean equations.

To the best of our knowledge, the only known generic method to compute the ANF of a Boolean function in r variables is the *binary Möbius transform*. It has complexity $r2^{r-1}$ (see e.g. [18]), independently of the degree or the number of monomials, and is only applicable if the look-up table can be fully computed and stored. This rules it out in almost all symmetric cryptographic contexts, including ours, since primitives are designed with a large enough codebook.

When the primitive can be algebraically described in a simple way, for instance as the composition of simple components (such as adder and multiplier logical circuits in the case of LWR), it is theoretically possible to directly compute the ANF as the composition of the ANF of each sub-functions, or to use alternative methods⁶ such as *monomial prediction* or *perfect division property* [40,13,28,27,10]. However, all these methods suffer from the same bottlenecks and become quickly impractical.

⁵ By Theorem 3, since the number of terms, degree and number of variables grow with m , this seems to be the most reasonable choice.

⁶ While these formalisms are well-suited to use generic MILP, SAT or CP solvers, estimating their memory and computational complexity remains also an open problem.

We show in the sequel that computing the ANF of $F^{m,n}$ – more precisely, a relevant *system of representatives* of the ANF – is in fact possible. To solve both memory and computational complexity issues and investigate the ANF for very large values of n , we rely on a further understanding of the structure of $F^{m,n}$. In Section 4, we show that $F^{m,n}$ inherits strong symmetries from commutativity of modular addition. We exploit those results to:

- reduce the study of the ANF of $F^{m,n}$ to the study of a relevant *system of representatives* which captures the *full ANF* up to permutations, and to
- reduce the study of $F^{m,n}$ for all values $m \leq n$ to the study of $F^{m,m}$.

The effective computation of the system of representatives of the ANF is described in Section 5.

Finally, we provide an in-depth study of the ANF in Section 6. By exploiting further the symmetries mentioned above, we compute additional metrics such as an improved upper bound on the dimension of the subspace spanned by the family of functions $\langle F_{\mathbf{a}}^{m,n}, \mathbf{a} \in (\mathbb{Z}_{2^q})^n \rangle$ and the average number of non-zero terms. As described in Section 7.1, this study allows us to systematically improve the linearisation attack of Arora and Ge.

4 Formalising symmetries in the ANF of $F^{m,n}$

In this section, we formalize the significant implications of two simple observations on the dot product.

1. By commutativity of modular addition, each pair of variables (a_i, x_i) plays an identical role. This creates strong symmetries in the ANF of (products of) bits of the dot product that are detailed in Lemma 2 and Theorem 4 below.
2. Furthermore, the dot product of n -long vectors can be seen as a dot product of n' -long vectors where $n \leq n'$ and where the last $n' - n$ coordinates are zeros. This implies that the ANF of $F^{m,n}$ is “included” in the ANF of $F^{m,n'}$. Theorem 5 below shows a converse property: it is actually sufficient to understand $F^{m,n}$ to fully understand $F^{m,n'}$.

To achieve such goals, we rely on the formalism of *group actions*. Standard definitions and results on group actions can e.g. be found in [22, Chapter 4].

4.1 Action of $\mathfrak{S}_n$ on Boolean functions

We denote by $\mathfrak{S}_n$ the symmetric group over the set $\{0, \dots, n-1\}$ and the composition of two permutations $\sigma, \tau \in \mathfrak{S}_n$ by $\sigma \circ \tau$ or $\sigma\tau$.

Action on vectors. Let E be a set. For any vector $\mathbf{u} \in E^n$ and any permutation $\sigma \in \mathfrak{S}_n$, we denote by $\sigma \cdot \mathbf{u}$, the permuted copy of $\mathbf{u}$ with respect to σ , namely:

$$\sigma \cdot \mathbf{u} := (u_{\sigma^{-1}(0)}, \dots, u_{\sigma^{-1}(n-1)}).$$

This corresponds to an action of the group $\mathfrak{S}_n$ on the set E^n . Similarly, $\mathfrak{S}_n$ acts on pairs of vectors and we denote by $\sigma \cdot (\mathbf{u}, \mathbf{v})$ the pair $(\sigma \cdot \mathbf{u}, \sigma \cdot \mathbf{v})$ for any n -long vectors $\mathbf{u}, \mathbf{v}$ and any permutation $\sigma \in \mathfrak{S}_n$.

Action on monomials and functions. The group $\mathfrak{S}_n$ also acts on monomials by permuting input variables. This is clearly equivalent to applying the inverse permutation to the vectors of exponents and we denote:

$$\sigma \cdot (\mathbf{x}^{\mathbf{v}}) := \mathbf{x}^{\sigma^{-1} \cdot \mathbf{v}} = (\sigma \cdot \mathbf{x})^{\mathbf{v}} \text{ and } \sigma \cdot (\mathbf{a}^{\mathbf{u}} \mathbf{x}^{\mathbf{v}}) := \mathbf{a}^{\sigma^{-1} \cdot \mathbf{u}} \mathbf{x}^{\sigma^{-1} \cdot \mathbf{v}} = (\sigma \cdot \mathbf{a})^{\mathbf{u}} (\sigma \cdot \mathbf{x})^{\mathbf{v}}.$$

More generally, for any Boolean function f over $\mathbb{Z}_{2^q}^n$ we define $\sigma \cdot f$ as:

$$\sigma \cdot f := f(\sigma \cdot \mathbf{x}) = \sum_{\mathbf{u} \in \mathbb{N}^n} \alpha_{\mathbf{u}} (\sigma \cdot \mathbf{x})^{\mathbf{u}} = \sum_{\mathbf{u} \in \mathbb{N}^n} \alpha_{\mathbf{u}} \mathbf{x}^{\sigma^{-1} \cdot \mathbf{u}} = \sum_{\mathbf{u} \in \mathbb{N}^n} \alpha_{\sigma \cdot \mathbf{u}} \mathbf{x}^{\mathbf{u}}; \quad (3)$$

For a function F over $\mathbb{Z}_{2^q}^n \times \mathbb{Z}_{2^q}^n$, $\sigma \cdot F$ can be seen as:

$$\sigma \cdot F = \sum_{\mathbf{u}, \mathbf{v} \in \mathbb{N}^n} \alpha_{\sigma \cdot (\mathbf{u}, \mathbf{v})} \mathbf{a}^{\mathbf{u}} \mathbf{x}^{\mathbf{v}} \in \mathbb{F}_2[\mathbf{a}, \mathbf{x}], \quad \text{or as} \quad (4)$$

$$\sigma \cdot F = \sum_{\mathbf{v} \in \mathbb{N}^n} (\sigma \cdot \alpha_{\mathbf{v}}) (\sigma \cdot \mathbf{x})^{\mathbf{v}} = \sum_{\mathbf{v} \in \mathbb{N}^n} (\sigma \cdot \alpha_{\sigma \cdot \mathbf{v}}) \mathbf{x}^{\mathbf{v}} \in \mathbb{F}_2[\mathbf{a}][\mathbf{x}]. \quad (5)$$

Example 4. Let $\mathbf{x} = (x_0, x_1)$, let σ be the 2-cycle $\sigma = (0 \ 1)$ and let $f: \mathbb{Z}_{2^1}^2 \rightarrow \mathbb{F}_2$ be defined by $f(\mathbf{x}) = x_0^3 + x_1^2 = x_{0,0}x_{0,1} + x_{1,1}$. Then $(\sigma \cdot f)(\mathbf{x}) = x_1^3 + x_0^2 = x_{1,0}x_{1,1} + x_{0,1}$. $\triangleright$

Example 5. Let $\mathbf{x} = (x_0, x_1)$, $\mathbf{a} = (a_0, a_1)$, σ be the 2-cycle $\sigma = (0 \ 1)$ and let $F: \mathbb{Z}_{2^1}^2 \times \mathbb{Z}_{2^1}^2 \rightarrow \mathbb{F}_2$ be defined as $F(\mathbf{a}, \mathbf{x}) = x_0^3 + x_1^2 + a_1$. Then $(\sigma \cdot F)(\mathbf{a}, \mathbf{x}) = x_1^3 + x_0^2 + a_0$. $\triangleright$

For any of the actions of $\mathfrak{S}_n$ defined above on a set E (E being a set of vectors, monomials or polynomials), the well-known *orbit* and *stabiliser* of $e \in E$ are defined by:

$$\text{Orb}(e) := \{\sigma \cdot e, \sigma \in \mathfrak{S}_n\} \subset E \quad \text{and} \quad \text{Stab}(e) := \{\sigma \in \mathfrak{S}_n \text{ s.t. } \sigma \cdot e = e\} \subset \mathfrak{S}_n.$$

Note that the set of orbits $\{\text{Orb}(e) \mid e \in E\}$ is a partition of E . Therefore, we denote by $\sim$ the equivalence relation defined by $e \sim e'$ if and only if $e' \in \text{Orb}(e)$.

Invariance under group actions. Let $\mathbb{G}$ be a subgroup of $\mathfrak{S}_n$. A Boolean function f over $\mathbb{Z}_{2^q}^n$, is said to be $\mathbb{G}$ -invariant if $\sigma \cdot f = f$ for any $\sigma \in \mathbb{G}$ or equivalently if $\mathbb{G} \subset \text{Stab}(f)$. The same holds for functions over $\mathbb{Z}_{2^q}^n \times \mathbb{Z}_{2^q}^n$.

By identifying the coefficients of f (resp. F) with the ones of $\sigma \cdot f$ (resp. $\sigma \cdot F$) outlined in Eqs. (3) to (5), the uniqueness of the ANF provides the following equivalent definitions of $\mathbb{G}$ -invariance that will be useful to formalise the symmetries of $F^{m,n}$.

Lemma 1. Let $f: \mathbb{Z}_{2^q}^n \rightarrow \mathbb{F}_2$ be a Boolean function and $\mathbb{G}$ be a subgroup of $\mathfrak{S}_n$. Then f is $\mathbb{G}$ -invariant if and only if:

$$\forall \sigma \in \mathbb{G}, \forall \mathbf{u} \in \mathbb{N}^n, \quad \alpha_{\sigma \cdot \mathbf{u}}(f) = \alpha_{\mathbf{u}}(f).$$

Lemma 2. Let $F: \mathbb{Z}_{2^q}^n \times \mathbb{Z}_{2^q}^n \rightarrow \mathbb{F}_2$ be a Boolean function of $\mathbf{a}$ and $\mathbf{x}$ and $\mathbb{G}$ be a subgroup of $\mathfrak{S}_n$. The following statements are equivalent:

- (i) F is $\mathbb{G}$ -invariant.
- (ii) $\forall \sigma \in \mathbb{G}, \forall \mathbf{u}, \mathbf{v} \in \mathbb{N}^n, \quad \alpha_{\sigma \cdot (\mathbf{u}, \mathbf{v})}(F) = \alpha_{\mathbf{u}, \mathbf{v}}(F).$
- (iii) $\forall \sigma \in \mathbb{G}, \forall \mathbf{v} \in \mathbb{N}^n, \quad \sigma \cdot \alpha_{\mathbf{v}}(F) = \alpha_{\sigma^{-1} \cdot \mathbf{v}}(F).$

Finally, we emphasize that the action we consider permutes n tuples of q variables *without permuting the q elements of each tuple*. The definition of $\mathfrak{S}_n$ -invariance is thus more general than the well-known notion of symmetric polynomial [31, Def. 1.73] in nq variables: a symmetric polynomial in nq variables is $\mathfrak{S}_n$ -invariant, while the converse is not true in general.

Example 6. The function

$$\begin{aligned} f &: \mathbb{Z}_{2^1}^2 &\longrightarrow & \mathbb{F}_2 \\ \mathbf{x} = (x_0, x_1) &\longmapsto & x_{0,0}x_{0,1} + x_{1,0}x_{1,1} \end{aligned}$$

is $\mathfrak{S}_2$ -invariant since for any $\mathbf{x}$ and any $\sigma \in \mathfrak{S}_2$, $\sigma \cdot f(\mathbf{x}) = f(\mathbf{x})$. Indeed $f(\mathbf{x}) = x_0^3 + x_1^3 = x_1^3 + x_0^3 = (0 \ 1) \cdot f(\mathbf{x})$. However, it is not symmetric since $f(\mathbf{x}_{0,1}, x_{0,0}, \mathbf{x}_{1,1}, x_{1,0}) \neq f(\mathbf{x}_{1,1}, x_{0,0}, \mathbf{x}_{0,1}, x_{1,0})$. $\triangleright$

4.2 Symmetries in the ANF of the dot product

We can now capture the strong symmetries of the ANF of the (product of) bits of the dot product.

Theorem 4. Let $n, m \in \mathbb{N}^*$. Then $F^{m,n}$ is $\mathfrak{S}_n$ -invariant.

Proof. Let $\otimes$ stands for the coordinate-wise modular multiplication in $\mathbb{Z}_{2^q}$. The announced property is a direct consequence of the commutativity of modular addition. Indeed, for any $\sigma \in \mathfrak{S}_n$, $\text{SUM}^n(\mathbf{x}) = \text{SUM}^n(\sigma \cdot \mathbf{x})$, which implies that:

$$\begin{aligned} \langle \mathbf{a}, \mathbf{x} \rangle_n &= \text{SUM}^n(\mathbf{a} \otimes \mathbf{x}) = \text{SUM}^n(\sigma \cdot (\mathbf{a} \otimes \mathbf{x})) \\ &= \text{SUM}^n((\sigma \cdot \mathbf{a}) \otimes (\sigma \cdot \mathbf{x})) = \langle \sigma \cdot \mathbf{a}, \sigma \cdot \mathbf{x} \rangle_n. \end{aligned} \quad \square$$

By Lemma 2, the $\mathfrak{S}_n$ -invariance of $F^{m,n}$ implies that effective gains can be expected for both computing and storing the ANF of $F^{m,n}$.

- By Item (ii), for any $(\mathbf{u}, \mathbf{v})$, all coefficients (in $\mathbf{a}$ and $\mathbf{x}$) $\alpha_{\mathbf{z}, \mathbf{t}}(F^{m,n})$ with $(\mathbf{z}, \mathbf{t}) \in \text{Orb}(\mathbf{u}, \mathbf{v})$ are equal. This implies that investigating (and storing) a single representative for each orbit is sufficient to understand the full ANF.

- By Item (iii), the same holds for coefficients in $\mathbf{x}$ of $F^{m,n}$. In that case, if $\mathbf{v}, \mathbf{z} \in \mathbb{N}^n$ are in the same orbit, the coefficients $\alpha_{\mathbf{v}}(F^{m,n})$ and $\alpha_{\mathbf{z}}(F^{m,n})$ are not necessarily equal but can be computed one from the other by applying a permutation (or its inverse).

Example 7. The ANF of $F^{2,2}$ is given by :

$$F^{2,2} = \underbrace{\mathbf{a}^{(1,0)}\mathbf{x}^{(1,0)} + \mathbf{a}^{(0,1)}\mathbf{x}^{(0,1)}}_{\text{Orb}((1,0),(1,0))} + \underbrace{\mathbf{a}^{(1,1)}\mathbf{x}^{(1,1)}}_{\text{Orb}((1,1),(1,1))} + \underbrace{\mathbf{a}^{(1,0)}\mathbf{x}^{(2,0)} + \mathbf{a}^{(0,1)}\mathbf{x}^{(0,2)}}_{\text{Orb}((1,0),(2,0))} + \underbrace{\mathbf{a}^{(2,0)}\mathbf{x}^{(1,0)} + \mathbf{a}^{(0,2)}\mathbf{x}^{(0,1)}}_{\text{Orb}((2,0),(1,0))}.$$

We observe that a term $\mathbf{a}^{\mathbf{u}}\mathbf{x}^{\mathbf{v}}$ appears if and only if all terms in its orbit appear. Furthermore, the coefficient of $\mathbf{x}^{(1,0)}$ is $\alpha_{(1,0)} = \mathbf{a}^{(1,0)} + \mathbf{a}^{(2,0)}$ and it is not equal to the coefficient of $\mathbf{x}^{(0,1)}$, namely $\alpha_{(0,1)} = \mathbf{a}^{(0,1)} + \mathbf{a}^{(0,2)}$, despite the fact that $\sigma \cdot (1,0) = (0,1)$ where σ is the 2-cycle (01). However, it holds that $\sigma^{-1} \cdot \alpha_{(1,0)} = \alpha_{(0,1)}$. $\triangleright$

The next proposition provides two useful observations on the support of the exponents of $\text{Exp}(F^{m,n})$ that will be decisive to capture the relationship between the ANF of $F^{m,m}$ and $F^{m,n}$ for $n \geq m$.

Proposition 2. *Let $n, m \in \mathbb{N}^*$. Let $(\mathbf{u}, \mathbf{v}) \in \text{Exp}(F^{m,n})$. Then :*

- (i) *$\mathbf{u}$ and $\mathbf{v}$ share the same support: $\forall 0 \leq i < n, u_i = 0 \iff v_i = 0$.*
- (ii) *This support has cardinality smaller than m : $|\text{Supp}(\mathbf{u})| = |\text{Supp}(\mathbf{v})| \leq m$.*

Proof. By Theorem 1, $F^{m,n} = \pi_m \circ \langle \mathbf{a}, \mathbf{x} \rangle_n = \sum_{\mathbf{e} \in S_m^n} \prod_{i=0}^{n-1} (a_i \times x_i)^{c_i}$. By Theorem 2, the ANF of $(a_i \times x_i)^{c_i}$ only contains terms of the form $a_i^u x_i^v$ with $u \neq 0$ and $v \neq 0$. Indeed, for any $\mathbf{e} \in \mathbb{N}^q$, it holds that $g(\mathbf{e}) \neq 0$ if and only if $h(\mathbf{e}) \neq 0$ by definition of the functions g and h . Item (ii) can be seen as a consequence of Item (i) of Theorem 3. Indeed the coordinates of $\mathbf{v}$, which are positive integers, must sum to less than m so $\mathbf{v}$ cannot have strictly more than m non-zero coordinates. $\square$

Together with Proposition 2, the next theorem allows us to show that the ANF of $F^{m,n'}$ for any $n' \geq n \geq m$ can be built from the ANF of $F^{m,n}$.

Theorem 5 (Scaling of the ANF). *Let $m, n, n' \in \mathbb{N}^*$ with $n' \geq n \geq m$. Let $F: \mathbb{Z}_{2^q}^n \times \mathbb{Z}_{2^q}^n \rightarrow \mathbb{F}_2$ be a $\mathfrak{S}_n$ -invariant function that also satisfies Items (i) and (ii) of Proposition 2. Then there exists a unique $\mathfrak{S}_{n'}$ -invariant function $F': \mathbb{Z}_{2^q}^{n'} \times \mathbb{Z}_{2^q}^{n'} \rightarrow \mathbb{F}_2$ that satisfies Item (i), Item (ii) and:*

$$\forall \mathbf{a}, \mathbf{x} \in \mathbb{Z}_{2^q}^n, \quad F(\mathbf{a}, \mathbf{x}) = F'(\mathbf{a} \parallel \underbrace{0, \dots, 0}_{n'-n}, \mathbf{x} \parallel \underbrace{0, \dots, 0}_{n'-n}). \quad (6)$$

We call F' the n' -extension of F .

Proof. A simple procedure to prove the existence is the following: for each orbit of monomials that appear in the ANF of F , select a representative $\mathbf{a}^{\mathbf{u}}\mathbf{x}^{\mathbf{v}}$ and add $\mathbf{a}^{\tilde{\mathbf{u}}}\mathbf{x}^{\tilde{\mathbf{v}}}$ to the ANF of F' where $\tilde{\mathbf{u}} = (\mathbf{u}||0, \dots, 0)$ and $\tilde{\mathbf{v}} = (\mathbf{v}||0, \dots, 0)$. Note that $\mathbf{a}^{\tilde{\mathbf{u}}}\mathbf{x}^{\tilde{\mathbf{v}}}$ is equal to $\mathbf{a}^{\mathbf{u}}\mathbf{x}^{\mathbf{v}}$ (but the exponents are of length n' instead of n , with $n' - n$ trailing zeros). Then, add all terms in the orbit of $\mathbf{a}^{\tilde{\mathbf{u}}}\mathbf{x}^{\tilde{\mathbf{v}}}$. By construction, F' is thus $\mathfrak{S}_{n'}$ -invariant, but also satisfies the properties of Proposition 2 (since these properties are invariant up to permutations). It also satisfies the extension property: all terms of F appears in the ANF of F' . For the remaining terms of F' , all of them depend on at least one of the $n' - n$ last variables, fixing these variables to 0 then cancel them all. Regarding unicity, if F'' shares the same properties as F' , then $F' - F''$ is also $\mathfrak{S}_{n'}$ -invariant and satisfies properties (i) and (ii). Because $F' - F''$ is 0 for all values $(\mathbf{a}||0, \dots, 0, \mathbf{x}||0, \dots, 0)$, either all its terms involve one of the $n' - n$ last pairs (a_i, x_i) , or it is the zero function. But the first case is impossible since any $\mathbf{a}^{\mathbf{u}}\mathbf{x}^{\mathbf{v}}$ satisfies $\text{Supp}(\mathbf{u}) = \text{Supp}(\mathbf{v})$, and $|\text{Supp}(\mathbf{u})| = |\text{Supp}(\mathbf{v})| \leq m \leq n$ so there always exists $\sigma \in \mathfrak{S}_{n'}$ such that $\sigma \cdot \mathbf{a}^{\mathbf{u}}\mathbf{x}^{\mathbf{v}}$ has support within the first n positions. So $F' = F''$. $\square$

By Proposition 2, $F^{m,n}$ for any m, n satisfies properties (i) and (ii). Furthermore, $F^{m,n'}$ and $F^{m,n}$ for any $n' \geq n$ also satisfy Eq. (6) since the n -long dot product $\sum_{i=0}^{n-1} a_i x_i$ can just be seen as a particular case of n' -long dot product $\sum_{i=0}^{n'-1} a_i x_i$ where the last $n' - n$ a_i 's and x_i 's are set to 0. In particular, $F^{m,n'}$ is the n' -extension of $F^{m,n}$, and the constructive proof of existence given for Theorem 5 can be used in our context: when $m \leq n \leq n'$, the ANF of $F^{m,n'}$ can be fully deduced from the ANF of $F^{m,n}$, or, when $m = n$, from the ANF of $F^{m,m}$.

Example 8. The ANF of $F^{2,2}$ is given by :

$$F^{2,2} = \mathbf{a}^{(1,0)}\mathbf{x}^{(1,0)} + \mathbf{a}^{(0,1)}\mathbf{x}^{(0,1)} + \mathbf{a}^{(1,1)}\mathbf{x}^{(1,1)} + \underbrace{\mathbf{a}^{(1,0)}\mathbf{x}^{(2,0)}}_{(*)} + \mathbf{a}^{(0,1)}\mathbf{x}^{(0,2)} + \mathbf{a}^{(2,0)}\mathbf{x}^{(1,0)} + \mathbf{a}^{(0,2)}\mathbf{x}^{(0,1)}.$$

By Theorem 5, the fact that the monomial $\mathbf{a}^{(1,0)}\mathbf{x}^{(2,0)}$ appears in the ANF of $F^{2,2}$ implies that it also appears in the ANF of $F^{2,4}$ under the form $\mathbf{a}^{(1,0,0,0)}\mathbf{x}^{(2,0,0,0)} = \mathbf{a}^{(1,0)}\mathbf{x}^{(2,0)}$. It therefore implies that $\mathbf{a}^{(0,1)}\mathbf{x}^{(0,2)} = \mathbf{a}^{(0,1,0,0)}\mathbf{x}^{(0,2,0,0)}$ also appears in $F^{2,4}$, but also $\mathbf{a}^{(0,0,1,0)}\mathbf{x}^{(0,0,2,0)}$ and $\mathbf{a}^{(0,0,0,1)}\mathbf{x}^{(0,0,0,2)}$ which did not appear in $F^{2,2}$ but belongs to the orbit of $\mathbf{a}^{(1,0)}\mathbf{x}^{(2,0)}$ when we consider vectors of size 4 instead of 2. $\triangleright$

Combined together, Theorems 4 and 5 state that it is sufficient to study a system of representatives of the terms of $F^{m,m}$ to fully understand all terms of $F^{m,n}$ for any $m \leq n$. In a similar manner, one can prove that studying the terms of support of size d of $F^{m,d}$ is sufficient to understand all terms of support of size d in $F^{m,n}$ with $d \leq n$. We show in Section 5 how we leverage these results to compute, store and analyze a system of representatives of the terms of $F^{m,m}$ in Python, on a 10-year-old standard laptop, in a few seconds for $m \leq 20$. The

relevance of storing and computing this succinct representation of the ANF is illustrated in Section 6, where we provide results such as the average number of non-zero terms or an upper-bound on the rank of the family of functions $(F_{\mathbf{a}}^{m,n})_{\mathbf{a} \in \mathbb{N}^n}$ for $m \leq 20$ and, most notably, for an arbitrary n .

5 Effective computation of a succinct representation of the ANF of $F^{m,n}$

In this section, we show how to effectively compute and store the full ANF up to permutations using the results of Section 4. By $\mathfrak{S}_n$ -invariance of $F^{m,n}$, we have shown that the coefficient $\alpha_{\mathbf{u},\mathbf{v}}$ of $\mathbf{a}^{\mathbf{u}}\mathbf{x}^{\mathbf{v}}$ is equal to the coefficient of any monomial in $\text{Orb}(\mathbf{a}^{\mathbf{u}}\mathbf{x}^{\mathbf{v}})$. In Section 5.1, we show how to compute the value of the coefficient of a representative monomial of each orbit. Finally, in Section 5.2, we show how to compute a system of representatives of the ANF of $F^{m,n}$ in a suitable form for the cryptanalyst.

In the following, let $\mathcal{C}^n$ be a *system of representatives* of $\mathbb{N}^n / \sim$, that is, a set built by picking an arbitrary element in each equivalence class of $\mathbb{N}^n / \sim$, *i.e.* in each orbit. For any $\mathbf{v}$, we denote by $\mathbf{v}^*$ the representative of $\text{Orb}(\mathbf{v})$, that is, the unique element in $\text{Orb}(\mathbf{v}) \cap \mathcal{C}^n$. Similarly, let $\mathcal{D}^n$ be a system of representatives for $\mathbb{N}^n \times \mathbb{N}^n / \sim$ and $(\mathbf{u}, \mathbf{v})^*$ be the unique element of $\text{Orb}(\mathbf{u}, \mathbf{v}) \cap \mathcal{D}^n$ for any $\mathbf{u}, \mathbf{v}$.

5.1 Computing a system of representatives of the $\alpha_{\mathbf{u},\mathbf{v}}$

Decomposing $F^{m,n}$ into sums and products of subfunctions. Recall that $S_m^n = \{\mathbf{c} \in \mathbb{N}^n \mid \sum_{i=0}^{n-1} c_i = m\}$. By Theorem 1, it holds that:

$$F^{m,n}(\mathbf{a}, \mathbf{x}) = \sum_{\mathbf{c} \in S_m^n} \prod_{i=0}^{n-1} (a_i \times x_i)^{c_i}.$$

Since permuting a vector preserves the sum of its coordinates, it is clear that S_m^n can be decomposed as a disjoint union of orbits and that $\mathcal{C}_m^n := \mathcal{C}^n \cap S_m^n$ contains a single representative of each of these orbits. In particular, one can write

$$F^{m,n}(\mathbf{a}, \mathbf{x}) = \sum_{\mathbf{c} \in \mathcal{C}_m^n} \sum_{\mathbf{c}' \in \text{Orb}(\mathbf{c})} \prod_{i=0}^{n-1} (a_i \times x_i)^{c'_i}.$$

We introduce some auxiliary functions $H_{\mathbf{c}}, G_{\mathbf{c}}$ for $\mathbf{c} \in \mathbb{N}^n$, that we define by:

$$H_{\mathbf{c}}(\mathbf{a}, \mathbf{x}) := \prod_{i=0}^{n-1} (a_i \times x_i)^{c_i} \quad \text{and} \quad G_{\mathbf{c}}(\mathbf{a}, \mathbf{x}) := \sum_{\mathbf{c}' \in \text{Orb}(\mathbf{c})} \prod_{i=0}^{n-1} (a_i \times x_i)^{c'_i},$$

so that one can express $F^{m,n}$ as:

$$F^{m,n}(\mathbf{a}, \mathbf{x}) = \sum_{\mathbf{c} \in \mathcal{C}_m^n} G_{\mathbf{c}}(\mathbf{a}, \mathbf{x}) = \sum_{\mathbf{c} \in \mathcal{C}_m^n} \sum_{\mathbf{c}' \in \text{Orb}(\mathbf{c})} H_{\mathbf{c}'}(\mathbf{a}, \mathbf{x}). \quad (7)$$

We first state important properties of these auxiliary functions.

Proposition 3. *Let $\mathbf{c} \in \mathbb{N}^n$. Then:*

- (i) *For any $\mathbf{z} \in \text{Orb}(\mathbf{c})$, $G_{\mathbf{z}} = G_{\mathbf{c}}$. Furthermore, $G_{\mathbf{c}}$ is $\mathfrak{S}_n$ -invariant.*
- (ii) *For any $\sigma \in \mathfrak{S}_n$, $\sigma \cdot H_{\mathbf{c}} = H_{\sigma^{-1} \cdot \mathbf{c}}$. In particular, for any $\mathbf{z} \in \text{Orb}(\mathbf{c})$, $H_{\mathbf{z}} \in \text{Orb}(H_{\mathbf{c}})$ and $H_{\mathbf{c}}$ is $\text{Stab}(\mathbf{c})$ -invariant.*

Proof. Let $\mathbf{z} \in \text{Orb}(\mathbf{c})$. The equality $G_{\mathbf{z}} = G_{\mathbf{c}}$ is an immediate consequence of the fact that $\text{Orb}(\mathbf{c}) = \text{Orb}(\mathbf{z})$. Before proving the $\mathfrak{S}_n$ -invariance of $G_{\mathbf{c}}$, we show (ii). It holds that:

$$\sigma \cdot H_{\mathbf{c}}(\mathbf{a}, \mathbf{x}) = \prod_{i=0}^{n-1} (a_{\sigma^{-1}(i)} \times x_{\sigma^{-1}(i)})^{c_i} = \prod_{j=0}^{n-1} (a_j \times x_j)^{c_{\sigma(j)}} = H_{\sigma^{-1} \cdot \mathbf{c}}(\mathbf{a}, \mathbf{x}),$$

where the second equality is obtained using the change of variable $i = \sigma(j)$. The $\text{Stab}(\mathbf{c})$ -invariance of $H_{\mathbf{c}}$ is clear from the previous equality. Regarding the $\mathfrak{S}_n$ -invariance of $G_{\mathbf{c}}$, by the previous equalities, it holds that:

$$\begin{aligned} \sigma \cdot G_{\mathbf{c}}(\mathbf{a}, \mathbf{x}) &= \sum_{\mathbf{c}' \in \text{Orb}(\mathbf{c})} \sigma \cdot H_{\mathbf{c}'}(\mathbf{a}, \mathbf{x}) \\ &= \sum_{\mathbf{c}' \in \text{Orb}(\mathbf{c})} H_{\sigma^{-1} \cdot \mathbf{c}'}(\mathbf{a}, \mathbf{x}) = \sum_{\mathbf{c}' \in \text{Orb}(\mathbf{c})} H_{\mathbf{c}'}(\mathbf{a}, \mathbf{x}) = G_{\mathbf{c}}(\mathbf{a}, \mathbf{x}). \quad \square \end{aligned}$$

Theorem 6 below shows that the value of $\alpha_{\mathbf{u}, \mathbf{v}}(G_{\mathbf{c}})$ can be computed by considering the ANF of a *single* $H_{\mathbf{c}'}$, where $\mathbf{c}' \in \text{Orb}(\mathbf{c})$, instead of all elements in the orbit of $\mathbf{c}$, as the definition of $G_{\mathbf{c}}$ suggests.

Theorem 6. *Let $\mathbf{c}, \mathbf{u}, \mathbf{v} \in \mathbb{N}^n \times \mathbb{N}^n$. Let $\mathcal{E}_{\mathbf{c}}(\mathbf{u}, \mathbf{v}) := \text{Exp}(H_{\mathbf{c}}) \cap \text{Orb}(\mathbf{u}, \mathbf{v})$. For any $\sigma \in \mathfrak{S}_n$, it holds that:*

$$\alpha_{\mathbf{u}, \mathbf{v}}(G_{\mathbf{c}}) = \frac{|\mathcal{E}_{\mathbf{c}}(\mathbf{u}, \mathbf{v})| n!}{|\text{Stab}(\mathbf{c})| |\text{Orb}(\mathbf{u}, \mathbf{v})|} \pmod{2}. \quad (8)$$

Proof. For any $(\mathbf{u}, \mathbf{v})$ and any $\mathbf{c}$ we have:

$$\alpha_{\mathbf{u}, \mathbf{v}}(G_{\mathbf{c}}) = \sum_{\sigma \in \mathfrak{S}_n / \text{Stab}(\mathbf{c})} \alpha_{\mathbf{u}, \mathbf{v}}(H_{\sigma \cdot \mathbf{c}}) = \sum_{\sigma \in \mathfrak{S}_n / \text{Stab}(\mathbf{c})} \alpha_{\sigma^{-1} \cdot (\mathbf{u}, \mathbf{v})}(H_{\mathbf{c}}),$$

since for any $\sigma \in \mathfrak{S}_n$, it can be shown that $\alpha_{\mathbf{u}, \mathbf{v}}(H_{\sigma \cdot \mathbf{c}}) = \alpha_{\sigma^{-1} \cdot (\mathbf{u}, \mathbf{v})}(H_{\mathbf{c}})$ using Proposition 3. Therefore it holds that:

$$\begin{aligned} |\text{Stab}(\mathbf{c})| \alpha_{\mathbf{u}, \mathbf{v}}(G_{\mathbf{c}}) &= \sum_{\tau \in \text{Stab}(\mathbf{c})} \alpha_{\mathbf{u}, \mathbf{v}}(G_{\tau \cdot \mathbf{c}}) \\ &= \sum_{\tau \in \text{Stab}(\mathbf{c})} \sum_{\sigma \in \mathfrak{S}_n / \text{Stab}(\tau \cdot \mathbf{c})} \alpha_{\sigma^{-1} \cdot (\mathbf{u}, \mathbf{v})}(H_{\tau \cdot \mathbf{c}}) \\ &= \sum_{\tau \in \text{Stab}(\mathbf{c})} \sum_{\sigma \in \mathfrak{S}_n / \text{Stab}(\mathbf{c})} \alpha_{\tau^{-1} \sigma^{-1} \cdot (\mathbf{u}, \mathbf{v})}(H_{\mathbf{c}}) \\ &= \sum_{\varphi \in \mathfrak{S}_n} \alpha_{\varphi \cdot (\mathbf{u}, \mathbf{v})}(H_{\mathbf{c}}), \end{aligned}$$

where we use again Proposition 3 for the third equality, but also the fact that $\text{Stab}(\tau \cdot \mathbf{c}) = \text{Stab}(\mathbf{c})$ since $\tau \in \text{Stab}(\mathbf{c})$. Finally, we obtain

$$\begin{aligned} \sum_{\varphi \in \mathfrak{S}_n} \alpha_{\varphi \cdot (\mathbf{u}, \mathbf{v})}(H_{\mathbf{c}}) &= \frac{n!}{|\text{Orb}(\mathbf{u}, \mathbf{v})|} \sum_{(\mathbf{z}, \mathbf{t}) \in \text{Orb}(\mathbf{u}, \mathbf{v})} \alpha_{(\mathbf{z}, \mathbf{t})}(H_{\mathbf{c}}) \\ &= \frac{n!}{|\text{Orb}(\mathbf{u}, \mathbf{v})|} |\mathcal{E}_{\mathbf{c}}(\mathbf{u}, \mathbf{v})|. \quad \square \end{aligned}$$

Efficient computing a system of representatives of the $\alpha_{\mathbf{u}, \mathbf{v}}$'s. The previous results enable us to simplify the computation of the ANF of $F^{m, n}$. By Equation 7, the ANF of $F^{m, n}$ is the sum of all $G_{\mathbf{c}}, \mathbf{c} \in \mathcal{C}_m^n$. By Theorem 6, any coefficient of $G_{\mathbf{c}}$ can be efficiently determined by considering the ANF of a single $H_{\mathbf{c}'}, \mathbf{c}' \in \text{Orb}(\mathbf{c})$. Finally, by $\mathfrak{S}_n$ -invariance of $G_{\mathbf{c}}$ (Proposition 3), it is sufficient to determine a single coefficient $\alpha_{\mathbf{u}, \mathbf{v}}(G_{\mathbf{c}})$ per orbit to determine whether the entire orbit appears or vanishes. Based on these results, we devise two algorithms.

1. First, Algorithm 1 computes a succinct but comprehensive description of the ANF of $G_{\mathbf{c}}$ from the ANF of $H_{\mathbf{c}'}, \mathbf{c}' \in \text{Orb}(\mathbf{c})$. More precisely, it returns the set $\text{Exp}_{\mathbf{a}, \mathbf{x}}^*(G_{\mathbf{c}}) := \{(\mathbf{u}, \mathbf{v})^* \in \mathcal{D}^n \mid \alpha_{(\mathbf{u}, \mathbf{v})^*}(G_{\mathbf{c}}) = 1\}$, which contains a single exponent for each appearing orbit. The initial computing of the ANF of $H_{\mathbf{c}}$ can be done using a direct and efficient procedure described in Appendix A.1.
2. Secondly, Algorithm 2 internally uses Algorithm 1 to determine the value of the set $\text{Exp}_{\mathbf{a}, \mathbf{x}}^*(F^{m, n}) := \{(\mathbf{u}, \mathbf{v})^* \in \mathcal{D}^n \mid \alpha_{(\mathbf{u}, \mathbf{v})^*}(F^{m, n}) = 1\}$, which again contains a single representative per orbit. As implied by the $\mathfrak{S}_n$ -invariance of $F^{m, n}$ (Theorem 4, Lemma 2), this fully determines all the coefficients of the ANF of $F^{m, n}$.

All the subroutines of our algorithms are described in Appendix A.1.

Algorithm 1 <code>Compute_Exp_star_Gc</code>	$\triangleright$ Computing $\text{Exp}_{\mathbf{a}, \mathbf{x}}^*(G_{\mathbf{c}})$.
---	--

```

1: Initialize an empty dictionary MAP and an array Exp_star_Gc.
2: Exp_Hc  $\leftarrow$  Compute_Exp_Hc  $\triangleright \text{Exp}_{\mathbf{a}, \mathbf{x}}(H_{\mathbf{c}})$ 
3: for all  $(\mathbf{u}, \mathbf{v}) \in \text{Exp\_Hc}$  do
4:   Compute  $(\mathbf{u}, \mathbf{v})^*$ 
5:   MAP $[(\mathbf{u}, \mathbf{v})^*]$   $\mathrel{+}= 1$ 
6: end for  $\triangleright \text{MAP}[(\mathbf{u}, \mathbf{v})^*] = |\mathcal{E}_{\mathbf{c}}((\mathbf{u}, \mathbf{v})^*)|$ 
7: for all  $(\mathbf{z}, \mathbf{t})$  such that MAP $[(\mathbf{z}, \mathbf{t})] \neq 0$  do
8:   if MAP $[(\mathbf{z}, \mathbf{t})] \cdot n! / (|\text{Stab}(\mathbf{c})| \cdot |\text{Orb}(\mathbf{z}, \mathbf{t})|) \equiv 1 \pmod{2}$  then
9:     Append  $(\mathbf{z}, \mathbf{t})$  to Exp_star_Gc
10:   end if
11: end for
12: return Exp_star_Gc.  $\triangleright \text{Exp}_{\mathbf{a}, \mathbf{x}}^*(G_{\mathbf{c}})$ 

```

Algorithm 2 Compute_Exp_star_Fmn ▷ Computing $\text{Exp}_{\mathbf{a}, \mathbf{x}}^*(F^{m,n})$.

```

1: Initialize an empty dictionary Coeff_Fmn and an empty array Exp_star_Fmn
2: for all  $c \in \mathcal{C}_m^n$  do
3:   Exp_star_Gc  $\leftarrow$  Compute_Exp_star_Gc( $c$ ) ▷ Algorithm 1.
4:   for all  $(\mathbf{u}, \mathbf{v})$  in Exp_star_Gc do
5:     Coeff_Fmn $[(\mathbf{u}, \mathbf{v})]$   $+=$  1
6:   end for
7: end for
8: for all  $(\mathbf{u}, \mathbf{v})$  in Coeff_Fmn do
9:   if Coeff_Fmn $[(\mathbf{u}, \mathbf{v})] \equiv 1 \pmod{2}$  then
10:    Append  $(\mathbf{u}, \mathbf{v})$  to Exp_star_Fmn
11:   end if
12: end for
13: return Exp_star_Fmn ▷  $\text{Exp}_{\mathbf{a}, \mathbf{x}}^*(F^{m,n})$ .

```

5.2 A more convenient representation for the cryptanalyst

The algorithms of the previous section allow us to obtain a succinct description of the ANF of $F^{m,n}$ as a polynomial in $\mathbf{a}$ and in $\mathbf{x}$ – a system of representatives of the exponents $(\mathbf{u}, \mathbf{v})$ such that $\mathbf{a}^{\mathbf{u}}\mathbf{x}^{\mathbf{v}}$ appears in the ANF (*i.e.*, $\alpha_{\mathbf{u}, \mathbf{v}} = 1$). However, this representation is not well-suited to the cryptanalyst because the variables $\mathbf{a}$ and $\mathbf{x}$ play very different roles: the variables $\mathbf{a}$ are *public* but *random*, whilst the variables $\mathbf{x}$ are secret. As detailed at the end of Section 2.4, a more convenient representation is to view $F^{m,n}$ as a polynomial in $\mathbf{x}$, with each monomial $\mathbf{x}^{\mathbf{v}}$ having a coefficient $\alpha_{\mathbf{v}} \in \mathbb{F}_2[\mathbf{a}]$. Indeed, this allows us to efficiently compute the polynomial $F_{\mathbf{a}}^{m,n}$ in $\mathbf{x}$ for any random $\mathbf{a}$.

Recall that $\text{Exp}_{\mathbf{x}}(F^{m,n}) = \{\mathbf{v} \in \mathbb{N}^n, \alpha_{\mathbf{v}}(F^{m,n}) \neq 0\}$. A priori, the goal of this section is thus to compute the $|\text{Exp}_{\mathbf{x}}(F^{m,n})|$ different $\alpha_{\mathbf{v}}$ for all $\mathbf{v} \in \text{Exp}_{\mathbf{x}}(F^{m,n})$. However, as before, we can exploit the $\mathfrak{S}_n$ -invariance of $F^{m,n}$: for any $\mathbf{v}$ and any $\mathbf{v}' \in \text{Orb}(\mathbf{v})$ such that $\sigma \cdot \mathbf{v}' = \mathbf{v}$, the coefficient $\alpha_{\mathbf{v}'}$ can be computed from $\alpha_{\mathbf{v}}$ as $\sigma \cdot \alpha_{\mathbf{v}} = \alpha_{\sigma^{-1} \cdot \mathbf{v}} = \alpha_{\mathbf{v}'}$ by Item (iii) of Lemma 2. In this section, we thus aim at computing the following set:

$$\text{Pairs}_{\mathbf{x}}(F^{m,n}) := \{(\mathbf{v}^*, \alpha_{\mathbf{v}^*}) : \mathbf{v}^* \in \text{Exp}_{\mathbf{x}}^*(F^{m,n})\}.$$

where $\text{Exp}_{\mathbf{x}}^*(F^{m,n}) := \mathcal{C}^n \cap \text{Exp}_{\mathbf{x}}(F^{m,n}) = \{\mathbf{v}^* \in \mathcal{C}^n, \alpha_{\mathbf{v}^*}(F^{m,n}) \neq 0\}$.

Remark 2. We emphasize that we purposefully consider a set of pairs. Indeed, we do not only want exponents as the cryptanalyst must know the ANF of each coefficient $\alpha_{\mathbf{v}}$ to build the linearised system. Furthermore, we do not only want a system of representatives of the coefficients, as in this case, we lose some important information to build the system:⁷ there may exist $\mathbf{v}_1^* \neq \mathbf{v}_2^*$ such that $\alpha_{\mathbf{v}_1^*} = \alpha_{\mathbf{v}_2^*}$ or, more generally, $\mathbf{v}_1^* \neq \mathbf{v}_2^*$ such that $\alpha_{\mathbf{v}_1^*} \in \text{Orb}(\alpha_{\mathbf{v}_2^*})$.

⁷ In fact, even in Section 6.2 where we exploit the fact that some monomials in $\mathbf{x}$ have the same coefficient, we need to know which ones in order to compute the new linearisation family.

The following proposition enables us to derive $\text{Pairs}_{\mathbf{x}}(F^{m,n})$ from the set $\text{Exp}_{\mathbf{x}}^*(F^{m,n})$ output by Algorithm 2.

Proposition 4. *Let $\mathbf{v} \in \mathbb{N}^n$. For any $\mathbf{t} \in \text{Orb}(\mathbf{v})$, let $\sigma_{\mathbf{t}}$ be a permutation such that $\sigma_{\mathbf{t}} \cdot \mathbf{t} = \mathbf{v}$. It holds that:*

$$\alpha_{\mathbf{v}} = \sum_{(\mathbf{z}, \mathbf{t}) \in \text{Exp}_{\mathbf{a}, \mathbf{x}}^*(F^{m,n}) \mid \mathbf{t} \in \text{Orb}(\mathbf{v})} \sum_{\tau \in \text{Stab}(\mathbf{v})} \mathbf{a}^{\tau \sigma_{\mathbf{t}} \cdot \mathbf{z}}.$$

Proof. It holds that:

$$\alpha_{\mathbf{v}} = \sum_{(\mathbf{z}, \mathbf{t}) \in \text{Exp}_{\mathbf{a}, \mathbf{x}}^*(F^{m,n})} \sum_{(\mathbf{u}, \mathbf{v}) \in \text{Orb}(\mathbf{z}, \mathbf{t})} \mathbf{a}^{\mathbf{u}}.$$

Let $(\mathbf{z}, \mathbf{t}) \in \text{Exp}_{\mathbf{a}, \mathbf{x}}^*(F^{m,n})$ such that $\mathbf{t} \in \text{Orb}(\mathbf{v})$ and let $\mathcal{P}$ be the projection $\mathcal{P} : (\mathbf{u}, \mathbf{v}) \mapsto \mathbf{u}$. Since $\text{Orb}(\mathbf{z}) = \mathcal{P}(\text{Orb}(\mathbf{z}, \mathbf{t}))$, there exists $\mathbf{u}$ such that $(\mathbf{z}, \mathbf{t}) \in \text{Orb}(\mathbf{u}, \mathbf{v})$, which is equivalent to $(\mathbf{u}, \mathbf{v}) \in \text{Orb}(\mathbf{z}, \mathbf{t})$. Moreover, for all such $\mathbf{u}$, $\text{Orb}(\mathbf{u}, \mathbf{v}) \cap \text{Exp}_{\mathbf{a}, \mathbf{x}}^*(F^{m,n}) \neq \emptyset$ which implies that $\alpha_{\mathbf{u}, \mathbf{v}} = 1$. Letting $\sigma_{\mathbf{t}}$ to be a permutation such that $\sigma_{\mathbf{t}} \cdot \mathbf{t} = \mathbf{v}$, we show that

$$\{\mathbf{u}, (\mathbf{u}, \mathbf{v}) \in \text{Orb}(\mathbf{z}, \mathbf{t})\} = \{\tau \sigma_{\mathbf{t}} \cdot \mathbf{z}, \tau \in \text{Stab}(\mathbf{v})\}.$$

Let $\tau \in \text{Stab}(\mathbf{v})$. It holds that $(\tau \sigma_{\mathbf{t}})^{-1}(\tau \sigma_{\mathbf{t}} \cdot \mathbf{z}, \mathbf{v}) = (\mathbf{z}, \mathbf{t})$, and thus $(\mathbf{z}, \mathbf{t}) \in \text{Orb}(\tau \sigma_{\mathbf{t}} \cdot \mathbf{z}, \mathbf{v})$. Conversely, let $\mathbf{u}$ such that $(\mathbf{z}, \mathbf{t}) \in \text{Orb}(\mathbf{u}, \mathbf{v})$. There exists σ such that $(\sigma \cdot \mathbf{u}, \sigma \cdot \mathbf{v}) = (\mathbf{z}, \mathbf{t})$. Since $\sigma^{-1} \cdot \mathbf{t} = \mathbf{v}$, $\sigma^{-1} \in \text{Stab}(\mathbf{v}) \sigma_{\mathbf{t}}$, and $\sigma^{-1} = \varphi \sigma_{\mathbf{t}}$ for some $\varphi \in \text{Stab}(\mathbf{v})$. Thus, $\mathbf{u} = \sigma^{-1} \cdot \mathbf{z} = \varphi \sigma_{\mathbf{t}} \cdot \mathbf{z}$. $\square$

From Proposition 4, we derive Algorithm 3 below that outputs $\text{Pairs}_{\mathbf{x}}(F^{m,n})$. All the implementation details and subroutines can be found in Appendix A.2.

Algorithm 3 Compute_Pairs_Fmn	▷ Computing $\text{Pairs}_{\mathbf{x}}(F^{m,n})$.
1: Initialize an empty dictionary Pairs_Fmn.	
2: Exp_star_Fmn ← Compute_Exp_star_Fmn.	▷ Algorithm 2.
3: for all $(\mathbf{u}, \mathbf{v}) \in \text{Exp_star_Fmn}$ do	
4: Compute $\tau \in \mathfrak{S}_n$ s.t. $\tau \cdot \mathbf{v} = \mathbf{v}^*$.	
5: Pairs_Fmn[$\mathbf{v}^*$] ← Pairs_Fmn[$\mathbf{v}^*$] $\cup \{\mathbf{a}^{\sigma \tau \cdot \mathbf{u}}, \sigma \in \text{Stab}(\mathbf{v}^*)\}$.	
6: end for	
7: return Pairs_Fmn.	▷ $\text{Pairs}_{\mathbf{x}}(F^{m,n})$ (as a dictionary).

6 Exploiting our succinct description of the full ANF

In the previous section, we presented how to take advantage of the symmetries of the ANF of $F^{m,n}$ to fully capture the ANF without having to fully store it

nor compute it. We implemented the algorithms described above in practice. They work well and fast on a standard computer for m up to 20. In this section, we show the relevancy of this succinct description of the ANF. As a main contribution, we show how to compute *exact* metrics about the Boolean non-linear system described in Section 3 for arbitrary large n *by only computing and storing a succinct description of the ANF for $n = m$* . The computation of these metrics can be seen as a proof of concept as it illustrates both the efficiency of our method and the relevance of the system of representatives. Moreover, as detailed in the sequel, they allow to assess further the security of LWR. In particular, our results show that we can improve the linearisation attack of Arora-Ge [6] beyond the natural benefit of working over $\mathbb{F}_2$ rather than $\mathbb{Z}_{2^q}$ (see Section 3.1) for all parameters considered, with various degrees of improvement.

6.1 Average number of non-zero terms in an equation

Many linear algebra algorithms take advantage of the sparsity of the involved matrices (e.g. the Block-Wiedemann algorithm [41,20]). It is thus interesting to measure the average number of non-zero terms in $\mathbf{x}$ in a Boolean LWR equation when $\mathbf{a}$ is uniformly random [26]. This corresponds to the sparsity of a random row in the linearisation matrix (see [26] for more details).

In our context, this quantity can be computed efficiently. Indeed, by $\mathfrak{S}_n$ -invariance of $F^{m,n}$, for any $\mathbf{v}^* \in \text{Exp}_{\mathbf{x}}^*(F^{m,n})$, there exist $|\text{Orb}(\mathbf{v}^*)|$ permuted copies of $\alpha_{\mathbf{v}^*}$ in the ANF of $F^{m,n}$, namely all $\alpha_{\mathbf{w}}$ for $\mathbf{w} \in \text{Orb}(\mathbf{v}^*)$. The bias of $\alpha_{\mathbf{v}^*}$ (or equivalently the probability $p_{\alpha_{\mathbf{v}^*}}$ that $\alpha_{\mathbf{v}^*}(\mathbf{a}) = 0$ when $\mathbf{a} \in \mathbb{Z}_{2^q}^n$ is drawn uniformly at random) remains unchanged when the inputs are permuted, and is therefore constant within the orbit of $\alpha_{\mathbf{v}^*}$ since $\alpha_{\sigma \cdot \mathbf{v}^*} = \sigma^{-1} \cdot \alpha_{\mathbf{v}^*}$. All in all, over uniformly random $\mathbf{a}$, the expected number of terms in $\mathbf{x}$ in the ANF of $F_{\mathbf{a}}^{m,n}(\mathbf{x})$ is given by:

$$\sum_{\mathbf{v}^* \in \text{Exp}_{\mathbf{x}}^*(F^{m,n})} (1 - p_{\alpha_{\mathbf{v}^*}}) |\text{Orb}(\mathbf{v}^*)|. \quad (9)$$

When the number of variables is small and the ANF is available, $p_{\alpha_{\mathbf{v}^*}}$ can be effectively computed e.g. through a (Fast) Fourier Transform. While Equation 9 depends on the length n of the vector (since the size of each orbit does), the next proposition states that we can easily extrapolate the results for $n > m$ from the case $n = m$. For any $\mathbf{v} \in \mathbb{N}^d$ such that $|\text{Supp}(\mathbf{v})| = d$ and for any $n \geq d$, we denote by $\text{Orb}_n(\mathbf{v})$ the orbit of $\mathbf{v}$ seen “as a vector of size n ”:

$$\text{Orb}_n(\mathbf{v}) := \left\{ \sigma \cdot \left(\mathbf{v} || \underbrace{(0, \dots, 0)}_{n-d} \right), \sigma \in \mathfrak{S}_n \right\}.$$

Let $\alpha \in \text{Coefficients}_{\mathbf{x}}(F^{m,n})$. By Proposition 2, for any $\mathbf{u} \in \text{Exp}_{\mathbf{a}}(\alpha)$ and any $\mathbf{v}$ such that $\alpha_{\mathbf{v}}(F^{m,n}) = \alpha$, it holds that $\text{Supp}(\mathbf{u}) = \text{Supp}(\mathbf{v})$ and thus that $\text{Supp}(\alpha) = \text{Supp}(\mathbf{v})$. For any $n \geq |\text{Supp}(\alpha)|$, we can thus define $\text{Orb}_n(\alpha)$ in a similar manner.

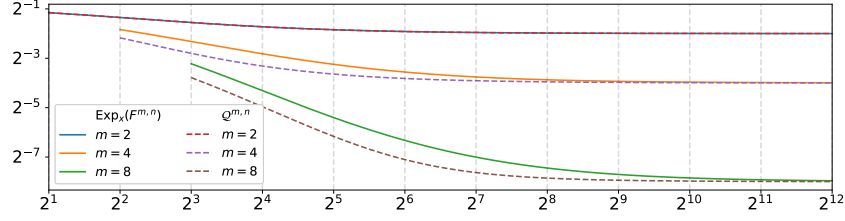


Fig. 1: Average fraction of terms in a random equation $F_{\mathbf{a}}^{m,n}(\mathbf{x})$ as a function of n , for $n \in \llbracket m, 4096 \rrbracket$ w.r.t the spanning families $\text{Exp}_{\mathbf{x}}(F^{m,n})$ and $\mathcal{Q}^{m,n}$.

Proposition 5. Let $\mathbf{v} \in \mathbb{N}^n$ and $d = |\text{Supp}(\mathbf{v})|$. Let $\alpha \in \text{Coefficients}_{\mathbf{x}}(F^{m,n})$, and $d' = |\text{Supp}(\alpha)|$. It holds that:

$$|\text{Orb}_n(\mathbf{v})| = |\text{Orb}_d(\mathbf{v})| \cdot \binom{n}{d} \quad \text{and} \quad |\text{Orb}_n(\alpha)| = |\text{Orb}_{d'}(\alpha)| \cdot \binom{n}{d'}.$$

Proof. Let I', I'' be two distinct subsets of $\llbracket 0, n-1 \rrbracket$ of cardinality d . The set $\{\mathbf{v}' \in \text{Orb}_n(\mathbf{v}), \text{Supp}(\mathbf{v}') = I'\}$ is of size exactly $|\text{Orb}_d(\mathbf{v})|$. It holds also that:

$$\{\mathbf{v}' \in \text{Orb}_n(\mathbf{v}), \text{Supp}(\mathbf{v}') = I'\} \cap \{\mathbf{v}'' \in \text{Orb}_n(\mathbf{v}), \text{Supp}(\mathbf{v}'') = I''\} = \emptyset.$$

The same holds for α since $\text{Supp}(\alpha) = \text{Supp}(\mathbf{v})$ for any $\mathbf{v}$ such that $\alpha_{\mathbf{v}} = \alpha$. $\square$

As a direct corollary, for $n > m$, the average sparsity can be efficiently and exactly computed from the case $n = m$ thanks to the following formula:

$$\sum_{\mathbf{v}^* \in \text{Exp}_{\mathbf{x}}^*(F^{m,m})} (1 - p_{\alpha_{\mathbf{v}^*}}) |\text{Orb}_{|\text{Supp}(\mathbf{v}^*)|}(\mathbf{v}^*)| \binom{n}{|\text{Supp}(\mathbf{v}^*)|}. \quad (10)$$

This is illustrated by the solid-lines of Figure 1; dashed lines are explained later.

6.2 Rank defect for LWR Boolean equations

Identifying a better linearising set. As already observed with Equation 1 of Section 2.4, expressing $F^{m,n}$ under the form $F^{m,n} = \sum_{\mathbf{v} \in \mathbb{N}^n} \alpha_{\mathbf{v}}(F^{m,n}) \mathbf{x}^{\mathbf{v}}$ allows the attacker to build an affine system $\mathcal{S}$ of the form

$$\begin{cases} F^{m,n}(\mathbf{a}^{(0)}, \mathbf{x}) &= y_0 \\ F^{m,n}(\mathbf{a}^{(1)}, \mathbf{x}) &= y_1 \\ &\vdots \\ F^{m,n}(\mathbf{a}^{(M-1)}, \mathbf{x}) &= y_{M-1} \end{cases}$$

where, for any i , $\mathbf{a}^{(i)}$ is the i -th random-input and y_i the corresponding output bit (or product of bits). Each equation is simply obtained by evaluating

all coefficients $\alpha_v(F^{m,n}) \in \mathbb{F}_2[\mathbf{a}]$ at point $\mathbf{a}^{(i)}$. In this context, an interesting metric to evaluate resistance to linearisation attacks is the *rank of the polynomial system*, that is, the dimension of the $\mathbb{F}_2$ -linear space V spanned by the set $\{F^{m,n}(\mathbf{a}^{(i)}, \mathbf{x})\}_{i \in \llbracket 0, M-1 \rrbracket}$ or, more generally, by the set $\{F_{\mathbf{a}}^{m,n}(\mathbf{x})\}_{\mathbf{a} \in \mathbb{Z}_{2^q}^n}$.

Assume for example that some monomials $\mathbf{x}^{v_0}, \dots, \mathbf{x}^{v_{s-1}}$ always appear and vanish together in the equations of $\mathcal{S}$. For a linearisation attack, it is then better to consider the polynomial $\sum_{i=0}^{s-1} \mathbf{x}^{v_i}$ as a single auxiliary variable instead of introducing s independent ones. Terms that appear and vanish at the same time depending on $\mathbf{a}$ are in fact terms which share the same coefficient $\alpha \in \text{Coefficients}_{\mathbf{x}}(F)$. We thus express $F^{m,n}$ as:

$$F^{m,n} = \sum_{\alpha \in \text{Coefficients}_{\mathbf{x}}(F^{m,n})} \alpha Q_{\alpha} \text{ where } Q_{\alpha} := \sum_{\substack{\mathbf{v} \in \mathbb{N}^n \text{ s.t.} \\ \alpha_v(F^{m,n}) = \alpha}} \mathbf{x}^{\mathbf{v}} \in \mathbb{F}_2[\mathbf{x}]. \quad (11)$$

The set of polynomials $\mathcal{Q}^{m,n} = \{Q_{\alpha}, \alpha \in \text{Coefficients}_{\mathbf{x}}(F^{m,n})\}$ is a generating set of V since, by design, any equation can be written as a linear combination of some Q_{α} . Its cardinality satisfies $|\mathcal{Q}^{m,n}| = |\text{Coefficients}_{\mathbf{x}}(F^{m,n})| \leq |\text{Exp}_{\mathbf{x}}(F^{m,n})| \leq \binom{n+m}{m} - 1$, which implies that it is a (strictly) smaller family than the family of the appearing monomials. When the first inequality is strict, a change of linearising set from $\text{Exp}_{\mathbf{x}}(F^{m,n})$ to $\mathcal{Q}^{m,n}$ improves the complexity of a linearisation attack from $|\text{Exp}_{\mathbf{x}}(F^{m,n})|^3$ to $|\mathcal{Q}^{m,n}|^3$, see e.g. [26] and the theoretical reason of the associated rank defect [7].

Remark 3. Note that $|\mathcal{Q}^{m,n}|$ is not the rank of the polynomial system, but a better upper bound on the rank than the number of monomials. Indeed, there may exist sets of coefficients that are linearly dependent but not all equal. Being able to compute the exact rank and an associated basis remains as an open question.

We managed to compute the set $\mathcal{Q}^{m,n}$ for values of $m \leq 20$ and n up to (an arbitrary value of) 4096. Our results are illustrated in Figure 2, where we compare $|\mathcal{Q}^{m,n}|$ to the number of monomials $|\text{Exp}_{\mathbf{x}}(F^{m,n})|$, the latter value being easily computed thanks to the results of Sections 4 and 5. As mentioned in Remark 1, in the case of a power-of-two m , we observed that $|\text{Exp}_{\mathbf{x}}(F^{m,n})| = \binom{n+m}{n} - 1$, matching the upper bound of Theorem 3.

Our upper bound on the rank suggests that the rank significantly drops compared to the number of monomials when n is small (when $m = 16$, $\frac{|\mathcal{Q}^{m,n}|}{|\text{Exp}_{\mathbf{x}}(F^{m,n})|} \simeq 0.22, 0.27, 0.45$ for $n = 64, 128, 256$ respectively) but could still tend to $\binom{n+m}{m} - 1$ for large values of n . We detail the gains provided by this drop in Section 7.1. For now, we describe our method for computing $\mathcal{Q}^{m,n}$.

Computing a system of representatives of the Q_{α} 's. In Theorem 7, we provide an orbit-by-orbit decomposition of each $Q_{\alpha} \in \mathbb{F}_2[\mathbf{x}]$ defined above and show that $Q_{\sigma \cdot \alpha} = \sigma \cdot Q_{\alpha}$. From there, our strategy will again consist in leveraging these structures to efficiently build a system of representatives of $\mathcal{Q}^{m,n} / \sim$ from the output of Algorithm 3 i.e., $\text{Pairs}_{\mathbf{x}}(F^{m,n}) := \{(\mathbf{v}^*, \alpha_{\mathbf{v}^*}) \mid \mathbf{v}^* \in \text{Exp}_{\mathbf{x}}^*(F^{m,n})\}$.

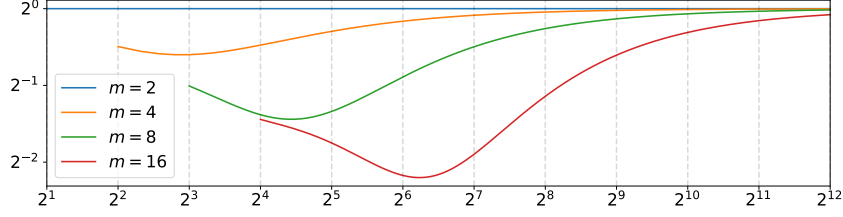


Fig. 2: $\frac{|Q^{m,n}|}{|\text{Exp}_x(F^{m,n})|}$ as a function of n , for $n \in \llbracket m, 4096 \rrbracket$.

Theorem 7. Let $n \in \mathbb{N}^*$, $\alpha \in \text{Coefficients}_x(F^{m,n})$. Let $\mathcal{B}_\alpha^n$ be defined as $\mathcal{B}_\alpha^n := \{\mathbf{w} \in \text{Exp}_x^*(F^{m,n}) \mid \alpha_{\mathbf{w}} \in \text{Orb}(\alpha)\}$. Then, it holds that:

$$(i) \ Q_\alpha^n = \sum_{\mathbf{v} \in \mathcal{B}_\alpha^n} \sum_{\substack{\mathbf{w} \in \text{Orb}(\mathbf{v}) \\ \alpha_{\mathbf{w}} = \alpha}} \mathbf{x}^{\mathbf{w}} \quad \text{and} \quad (ii) \text{ for any } \sigma \in \mathfrak{S}_n, \ Q_{\sigma \cdot \alpha}^n = \sigma \cdot Q_\alpha^n.$$

Proof. Item (i) holds because $F^{m,n}(\mathbf{a}, \mathbf{x}) = \sum_{\mathbf{v} \in \text{Exp}_x^*(F^{m,n})} \sum_{\mathbf{w} \in \text{Orb}(\mathbf{v})} \mathbf{x}^{\mathbf{w}} \alpha_{\mathbf{w}}$ by $\mathfrak{S}_n$ -invariance of $F^{m,n}$. Item (ii) stems from $\mathcal{B}_\alpha^n = \mathcal{B}_{\sigma \cdot \alpha}^n$ and:

$$\sigma \cdot \sum_{\substack{\mathbf{w} \in \text{Orb}(\mathbf{v}) \\ \alpha_{\mathbf{w}} = \alpha}} \mathbf{x}^{\mathbf{w}} = \sum_{\substack{\mathbf{w} \in \text{Orb}(\mathbf{v}) \\ \alpha_{\mathbf{w}} = \alpha}} \mathbf{x}^{\sigma^{-1} \cdot \mathbf{w}} = \sum_{\substack{\mathbf{t} \in \text{Orb}(\mathbf{v}) \\ \alpha_{\sigma \cdot \mathbf{t}} = \alpha}} \mathbf{x}^{\mathbf{t}} = \sum_{\substack{\mathbf{t} \in \text{Orb}(\mathbf{v}) \\ \sigma^{-1} \cdot \alpha_{\mathbf{t}} = \alpha}} \mathbf{x}^{\mathbf{t}} = \sum_{\substack{\mathbf{t} \in \text{Orb}(\mathbf{v}) \\ \alpha_{\mathbf{t}} = \sigma \cdot \alpha}} \mathbf{x}^{\mathbf{t}}.$$

□

Algorithm 4, of which the pseudo-code is provided in Appendix B, thus enables us to compute a system of representatives of $Q^{m,n} / \sim$ using the following strategy. For each $\alpha_{\mathbf{v}^*}$ output by Algorithm 3, we efficiently go over $\text{Orb}(\alpha_{\mathbf{v}^*})$ by leveraging the fact that, since $F^{m,n}$ is $\mathfrak{S}_n$ -invariant, $\sigma \cdot \alpha_{\mathbf{v}^*}(F) = \alpha_{\sigma^{-1} \cdot \mathbf{v}^*}(F)$ for any σ and therefore $\alpha_{\mathbf{v}^*}$ is $\text{Stab}(\mathbf{v}^*)$ -invariant.⁸ Two things can happen while going through $\text{Orb}(\alpha_{\mathbf{v}^*})$:

- Either we detect $\alpha_{\sigma \cdot \mathbf{v}^*} \in \text{Orb}(\alpha_{\mathbf{v}^*})$ such that $\alpha_{\sigma \cdot \mathbf{v}^*} = \alpha_{\mathbf{w}^*}$ for some $\mathbf{w}^* \neq \mathbf{v}^*$ that we already scanned before. In this case, we add $\mathbf{x}^{\sigma \cdot \mathbf{v}^*}$ to $Q_{\mathbf{w}^*}$, as well as all monomials $\mathbf{x}^{\tau \cdot \mathbf{v}^*}$ that satisfy $\alpha_{\tau \cdot \mathbf{v}^*} = \alpha_{\mathbf{w}^*}$.
- If no such element exists in $\text{Orb}(\alpha_{\mathbf{v}^*})$, then $\alpha_{\mathbf{v}^*}$ lies in a new orbit that will have $\alpha_{\mathbf{v}^*}$ as representative. Thus we create an entry for $Q_{\alpha_{\mathbf{v}^*}}$, and add $\mathbf{x}^{\mathbf{v}^*}$ to it, as well as all monomials $\mathbf{x}^{\tau \cdot \mathbf{v}^*}$ that satisfy $\alpha_{\tau \cdot \mathbf{v}^*} = \alpha_{\mathbf{v}^*}$.

Since Q_α is defined in Equation 11 as the sum over all $\mathbf{x}^{\mathbf{v}}$ such that $\alpha_{\mathbf{v}} = \alpha$, Algorithm 4 also enables to immediately derive a system of representatives for the coefficients $\alpha_{\mathbf{v}}$'s.

⁸ This is in fact true for any coefficient $\alpha_{\mathbf{v}}$ of a $\mathfrak{S}_n$ -invariant Boolean function $F: \mathbb{Z}_{2^q}^n \times \mathbb{Z}_{2^q}^n \rightarrow \mathbb{F}_2$.

Scaling of the pairs (α, Q_α) . Because of the one-to-one correspondence between the α 's and Q_α 's, it is easy to compute the size of $|\mathcal{Q}^{m,n}|$ for an arbitrary $n \geq m$ by scaling the size of the orbits of each α_v in a system of representatives of the coefficients, namely $\text{Coefficients}_x(F^{m,m})/\sim$. We obtain the following formula:

$$|\mathcal{Q}^{m,n}| = \sum_{\alpha \in \text{Coefficients}_x(F^{m,m})/\sim} \text{Orb}_{|\text{Supp}(\alpha)|}(\alpha) \cdot \binom{n}{|\text{Supp}(\alpha)|}.$$

As illustrated in Figure 2, we computed the cardinality of $\mathcal{Q}^{m,m}$ for m up to 16 and scaled it for n up to (an arbitrary value of) 4096. It is also possible to compute the average number of non-zero coefficients $\alpha \in \text{Coefficients}_x(F^{m,m})$ for a uniformly random sample $\mathbf{a}$. It exactly corresponds to the sparsity of equations with respect to this new generating set and, similarly to Equation 10, it is given by:

$$\sum_{\alpha \in \text{Coefficients}_x(F^{m,m})/\sim} (1 - p_\alpha) \text{Orb}_{|\text{Supp}(\alpha)|}(\alpha) \binom{n}{|\text{Supp}(\alpha)|}.$$

We computed the average sparsity for m up to 8, and Figure 1 displays the value obtained through scaling for n up to (an arbitrary value of) 4096.

7 Conclusion and open problems

7.1 A systematic improvement over the attack of Arora and Ge

Our techniques allow to systematically improve the attack of Arora and Ge (or to launch a valid attack when the one of Arora and Ge is not applicable, see Section 2.2). In Table 1, we propose to compare the cost of a linearisation attack using the generic method of Arora-Ge, applicable to both LWE and LWR, with our work. As detailed in Section 2.3, the complexity of applying the Arora-Ge technique to LWR is at least $2^{q-p} \cdot \binom{n+2^{q-p}}{n}^\omega$.⁹ In our case, we look at the LSB of the sample, which corresponds to $m = 2^{q-p}$ since our theoretical and experimental results show that this yields the lowest complexities.¹⁰ We provide a comparison with Arora-Ge using two metrics. First, we compare with the cost improved thanks to the rank drop using the formula $|\mathcal{Q}^{m,n}|^\omega$ (see the dashed curves of Fig. 2). We emphasize that we verified that our generating family $\mathcal{Q}^{m,n}$ allows to recover the secret as it contains all degree-1 monomials. Both in the case of Arora-Ge and our method, we use $\omega = 3$. Next, we compare with the cost improved by additionally taking into account the average sparsity in this new representation. We estimate the cost to be $2^{-3.42} \cdot p_{F^{m,n}} \cdot |\mathcal{Q}^{m,n}|^3$ using the Block-Wiedemann algorithm [41,20] where $p_{F^{m,n}}$ is the average number of non-zero terms per equation in the new linearisation family (see Fig. 1).

⁹ An Arora-Ge system has at most $\binom{n+m}{n}$ monomials in $\mathbf{x}$ and we showed in Section 2.3 that the smallest ring in which the solutions can be considered is $\mathbb{Z}_{2^{2q-p}}$.

¹⁰ Moreover, as detailed in Section 3.3, solving the system for the LSB allows to recover the full secret with a negligible additional cost.

$m = 2^{q-p}$	n	Arora-Ge [6]	Our work (rank only)	Our work (rank and sparsity)
8	64	$2^{103.4}$	$2^{97.8}$	$2^{87.2}$
	128	$2^{126.3}$	$2^{121.8}$	$2^{110.8}$
	256	$2^{149.7}$	$2^{145.9}$	$2^{134.7}$
16	64	$2^{167.7}$	$2^{157.2}$	Non-available
	128	$2^{211.7}$	$2^{202.0}$	Non-available
	256	$2^{257.5}$	$2^{250.1}$	Non-available

Table 1: Comparison to the linearisation attack by Arora and Ge using $\omega = 3$.

$m = 2^{q-p}$	n	Arora-Ge [6]	Our work (rank only)
8	64	$2^{70.0}$	$2^{65.2}$
	128	$2^{85.2}$	$2^{81.2}$
	256	$2^{100.1}$	$2^{97.3}$
16	64	$2^{113.2}$	$2^{104.8}$
	128	$2^{142.4}$	$2^{134.6}$
	256	$2^{173.0}$	$2^{166.7}$

Table 2: Comparison to the linearisation attack by Arora and Ge using $\omega = 2$.

On the choice of ω . For our comparison, we used $\omega = 3$ as the linear algebra constant. This simple choice seemed fair in the sense that it has no hidden constants as compared to e.g. Strassen’s algorithm. For completeness and fairness, we propose another comparison using $\omega = 2$ in Table 2. This makes particular sense from a design perspective – it is the constant used in e.g. the lattice estimator by Albrecht et al. [4]. Since the square of the size of the linearization matrix is a strict lower bound on the complexity of Block-Wiedemann (see [26]), we do not compare with this method of resolution.

7.2 Open problems

In the following, we detail some interesting directions.

Explaining the rank defect. In Section 6.2, we identified a rank defect as $|\mathcal{Q}_{m,n}|$ yields a strictly tighter upper bound on the rank than the number of monomials. However, determining the exact rank of the family $\{F_{\mathbf{a}}^{m,n}(\mathbf{x})\}_{\mathbf{a} \in \mathbb{Z}_{2^q}^n}$ remains an open problem. Nonetheless, we point out some interesting behaviours that can be viewed as first steps towards a full understanding. The first observation is that $\mathfrak{S}_n$ -invariance, which stems from the commutativity of addition, does not cover the full symmetries of $F^{m,n}$. Indeed, commutativity of multiplication implies that $F^{m,n}(\mathbf{a}, \mathbf{x}) = F^{m,n}(\mathbf{x}, \mathbf{a})$, which can be shown to imply that $\mathbf{a}^u \mathbf{x}^v$ appears if and only if $\mathbf{a}^v \mathbf{x}^u$ appears too. In particular, if the polynomial $\alpha_v = \mathbf{a}^u + \dots$

contains the monomial $\mathbf{a}^{\mathbf{u}}$, then necessarily $\alpha_{\mathbf{u}} = \mathbf{a}^{\mathbf{v}} + \dots$ contains the monomial $\mathbf{a}^{\mathbf{v}}$. This symmetry can be observed with the ANF of $F^{2,2}$ given in Example 7.

In our experiments, we also often observe that, when $\alpha_{\mathbf{v}} = \mathbf{a}^{\mathbf{u}} + \mathbf{a}^{\mathbf{u}'} + \mathbf{a}^{\mathbf{u}''} + \dots$ admits more than one monomial, then $\alpha_{\mathbf{u}} = \alpha_{\mathbf{u}'} = \alpha_{\mathbf{u}''} = \mathbf{a}^{\mathbf{v}}$, inducing a rank defect. This phenomena can be observed on the graphical representation of the ANF of $F^{4,4}$, given in Figure 3 of Appendix C.¹¹

ANF of modular multiplication and tight number of terms. It also remains an open problem to find an exact formula for the ANF of modular addition in which, contrary to Theorem 2, terms cannot cancel each others. Furthermore, from our experiments, we conjecture the bound on the number of terms of $F^{m,n}$ given in Item (i) of Theorem 3 to be tight when m is a power of two.

Observations on the unbalanced role of each bit. Recall that, by Item (i) of Theorem 3, a necessary condition for $\mathbf{x}^{\mathbf{u}}$ to appear in the ANF of $F^{m,n}$ is that $\mathbf{u} \in \bigcup_{i=1}^m S_i^n$, i.e. that the coordinates of $\mathbf{u}$ sum to less than m . This constraint implies that for any i , the role of each variable $x_{i,0}, \dots, x_{i,q-1}$ is very different since the higher the bit index j is, the more “costly” $x_{i,j}$ is.

Example 9. Let $i \in \llbracket 0, n-1 \rrbracket$ and $m = 2^\ell$. The only monomials that depend on $x_{i,\ell}$ are the degree-1 monomials $x_{i,\ell}$ since the only corresponding $\mathbf{u} \in \bigcup_{i=1}^m S_i^n$ have the form $(2^\ell, 0, \dots, 0)$ (up to permutation).

On the other hand, the variables $x_{i,0}$ (which only ‘cost’ $2^0 = 1$) are present in monomials of any degree, since there is always an integer partition involving some ones. When $2^\ell \leq n$, a monomial of highest degree 2^ℓ is necessarily of the form $\mathbf{x}^{\mathbf{u}} = \prod_{i \in \mathcal{I}} x_{i,0}$, for some $\mathcal{I}$ s.t. $|\mathcal{I}| = 2^\ell$ and its coefficient is $\alpha_{\mathbf{u}} = \prod_{i \in \mathcal{I}} a_{i,0}$. $\triangleright$

On other solving strategies. In our case, we relied on a linearisation strategy to solve the polynomial systems obtained. Whilst we exploit some specificities of the system such as a rank drop and sparsity, a natural question is whether it would be possible to further improve our attack with other solving strategies. Gröbner bases, for example, could allow to attack with less samples, significantly improving the relevance of our attack in contexts with a bounded number of samples. Hybrid strategies, using guess and solve approaches, would also be interesting to consider. In this context, the clear asymmetries in the role played by $x_{i,0}, \dots, x_{i,q-1}$ for a fixed i may be beneficial to an adversary, for example:

- Guessing the LSBs $x_{i,0}$ seems more interesting than guessing more significant bits, e.g. because highest-degree monomials exclusively depend on them.
- From a dual point of view, values of $\mathbf{a}$ such that most a_i are even (i.e. $a_{i,0} = 0$) lead to strictly less high-degree monomials. Time-data trade-offs could thus be considered, and be even more efficient for variants such as Ring or Mod-LWR where such a property is spread over several equations.

¹¹ Note that whilst there are no counter example of the observation above in the ANF of $F^{4,4}$, there exist counter examples of the observation for greater values of m .

These non-exhaustive examples suggest that the Boolean modeling of LWR leads to new attack paths that deserve further investigation.

Acknowledgments. François-Xavier Standaert is a research director of the Belgian Fund for Scientific Research (F.R.S.-FNRS). This work has been funded in part by the ERC project 101096871 (acronym BRIDGE). Views and opinions expressed are those of the authors only and do not necessarily reflect those of the European Union or the European Research Council. Neither the European Union nor the granting authority can be held responsible for them. This work was also supported by project Cryptanalyse from PEPR Cybersécurité (22-PECY-0010).

References

1. Alagic, G., Apon, D., Cooper, D., Dang, Q., Dang, T., Kelsey, J., Lichtinger, J., Miller, C., Moody, D., Peralta, R., Perlner, R., Robinson, A., Smith-Tone, D., Liu, Y.K.: Status report on the third round of the nist post-quantum cryptography standardization process. Tech. Rep. NIST Internal Report (IR) 8413, Rev. 1, Includes updates as of September 9, 2022, National Institute of Standards and Technology, Gaithersburg, MD (2022). <https://doi.org/10.6028/NIST.IR.8413-upd1>
2. Alamati, N., Montgomery, H., Patranabis, S.: Symmetric primitives with structured secrets. In: Boldyreva, A., Micciancio, D. (eds.) CRYPTO 2019, Part I. LNCS, vol. 11692, pp. 650–679 (Aug 2019). https://doi.org/10.1007/978-3-030-26948-7_23
3. Albrecht, M.R., Cid, C., Faugère, J.C., Fitzpatrick, R., Perret, L.: Algebraic algorithms for LWE problems. ACM Commun. Comput. Algebra **49**(2), 62 (Aug 2015). <https://doi.org/10.1145/2815111.2815158>
4. Albrecht, M.R., Player, R., Scott, S.: On the concrete hardness of learning with errors. J. Math. Cryptol. **9**(3), 169–203 (2015). <https://doi.org/10.1515/jmc-2015-0016>
5. Alwen, J., Krenn, S., Pietrzak, K., Wichs, D.: Learning with rounding, revisited - new reduction, properties and applications. In: Canetti, R., Garay, J.A. (eds.) CRYPTO 2013, Part I. LNCS, vol. 8042, pp. 57–74 (Aug 2013). https://doi.org/10.1007/978-3-642-40041-4_4
6. Arora, S., Ge, R.: New algorithms for learning in presence of errors. In: Aceto, L., Henzinger, M., Sgall, J. (eds.) ICALP 2011, Part I. LNCS, vol. 6755, pp. 403–415. Springer, Berlin, Heidelberg (Jul 2011). https://doi.org/10.1007/978-3-642-22006-7_34
7. Bak, A.: A note on the rank defect phenomena in the linearization attack on elisabeth-4. Cryptology ePrint Archive, Paper 2025/1133 (2025), <https://eprint.iacr.org/2025/1133>
8. Banerjee, A., Brenner, H., Leurent, G., Peikert, C., Rosen, A.: SPRING: Fast pseudorandom functions from rounded ring products. In: Cid, C., Rechberger, C. (eds.) FSE 2014. LNCS, vol. 8540, pp. 38–57. Springer, Berlin, Heidelberg (Mar 2015). https://doi.org/10.1007/978-3-662-46706-0_3
9. Banerjee, A., Peikert, C., Rosen, A.: Pseudorandom functions and lattices. In: Pointcheval, D., Johansson, T. (eds.) EUROCRYPT 2012. LNCS, vol. 7237, pp. 719–737 (Apr 2012). https://doi.org/10.1007/978-3-642-29011-4_42
10. Beyne, T., Verbaudhede, M.: Integral cryptanalysis using algebraic transition matrices. IACR Trans. Symm. Cryptol. **2023**(4), 244–269 (2023). <https://doi.org/10.46586/tosc.v2023.i4.244-269>

11. Boneh, D., Lewi, K., Montgomery, H.W., Raghunathan, A.: Key homomorphic PRFs and their applications. In: Canetti, R., Garay, J.A. (eds.) CRYPTO 2013, Part I. LNCS, vol. 8042, pp. 410–428 (Aug 2013). https://doi.org/10.1007/978-3-642-40041-4_23
12. Bostan, A., Chyzak, F., Giusti, M., Lebreton, R., Lecerf, G., Salvy, B., Schost, E.: Algorithmes Efficaces en Calcul Formel. Frédéric Chyzak (self-edited), Palaiseau (Sep 2017), <https://hal.archives-ouvertes.fr/AECF/>, 686 pages.
13. Boura, C., Canteaut, A.: Another view of the division property. In: Robshaw, M., Katz, J. (eds.) CRYPTO 2016, Part I. LNCS, vol. 9814, pp. 654–682 (Aug 2016). https://doi.org/10.1007/978-3-662-53018-4_24
14. Boura, C., Canteaut, A., De Cannière, C.: Higher-order differential properties of Keccak and Luffa. In: Joux, A. (ed.) FSE 2011. LNCS, vol. 6733, pp. 252–269. Springer, Berlin, Heidelberg (Feb 2011). https://doi.org/10.1007/978-3-642-21702-9_15
15. Braeken, A., Semaev, I.: The ANF of the composition of addition and multiplication mod 2^n with a Boolean function. In: Gilbert, H., Handschuh, H. (eds.) FSE 2005. LNCS, vol. 3557, pp. 112–125. Springer, Berlin, Heidelberg (Feb 2005). https://doi.org/10.1007/11502760_8
16. Brakerski, Z., Gentry, C., Vaikuntanathan, V.: (Leveled) fully homomorphic encryption without bootstrapping. In: Goldwasser, S. (ed.) ITCS 2012. pp. 309–325. ACM (Jan 2012). <https://doi.org/10.1145/2090236.2090262>
17. Brakerski, Z., Langlois, A., Peikert, C., Regev, O., Stehlé, D.: Classical hardness of learning with errors. In: Boneh, D., Roughgarden, T., Feigenbaum, J. (eds.) 45th ACM STOC. pp. 575–584. ACM Press (Jun 2013). <https://doi.org/10.1145/2488608.2488680>
18. Canteaut, A.: Lecture notes on cryptographic boolean functions (2016)
19. Chuengsatiansup, C., Stehlé, D.: Towards practical GGM-based PRF from (module-)learning-with-rounding. In: Paterson, K.G., Stebila, D. (eds.) SAC 2019. LNCS, vol. 11959, pp. 693–713 (Aug 2019). https://doi.org/10.1007/978-3-030-38471-5_28
20. Coppersmith, D.: Solving homogeneous linear equations over $gf(2)$ via block wiedemann algorithm. *Mathematics of Computation* **62**(205), 333–350 (1994), <http://www.jstor.org/stable/2153413>
21. D’Anvers, J.P., Karmakar, A., Roy, S.S., Vercauteren, F.: Saber: Module-LWR based key exchange, CPA-secure encryption and CCA-secure KEM. In: Joux, A., Nitaj, A., Rachidi, T. (eds.) AFRICACRYPT 18. LNCS, vol. 10831, pp. 282–305 (May 2018). https://doi.org/10.1007/978-3-319-89339-6_16
22. Dummit, D.S., Foote, R.M.: Abstract Algebra, 3rd Edition. John Wiley & Sons Inc (2003)
23. Dziembowski, S., Faust, S., Herold, G., Journault, A., Masny, D., Standaert, F.X.: Towards sound fresh re-keying with hard (physical) learning problems. In: Robshaw, M., Katz, J. (eds.) CRYPTO 2016, Part II. LNCS, vol. 9815, pp. 272–301 (Aug 2016). https://doi.org/10.1007/978-3-662-53008-5_10
24. von zur Gathen, J., Gerhard, J.: Modern Computer Algebra (3. ed.). Cambridge University Press (2013)
25. Geihs, M., Montgomery, H.: Lakey: Efficient lattice-based distributed prfs enable scalable distributed key management. In: USENIX Security Symposium. USENIX Association (2024)
26. Gilbert, H., Boissier, R.H., Jean, J., Reinhard, J.R.: Cryptanalysis of elisabeth-4. In: Guo, J., Steinfeld, R. (eds.) ASIACRYPT 2023, Part III. LNCS, vol. 14440, pp. 256–284 (Dec 2023). https://doi.org/10.1007/978-981-99-8727-6_9

27. Hebborn, P., Lambin, B., Leander, G., Todo, Y.: Lower bounds on the degree of block ciphers. In: Moriai, S., Wang, H. (eds.) ASIACRYPT 2020, Part I. LNCS, vol. 12491, pp. 537–566 (Dec 2020). https://doi.org/10.1007/978-3-030-64837-4_18
28. Hu, K., Sun, S., Wang, M., Wang, Q.: An algebraic formulation of the division property: Revisiting degree evaluations, cube attacks, and key-independent sums. In: Moriai, S., Wang, H. (eds.) ASIACRYPT 2020, Part I. LNCS, vol. 12491, pp. 446–476 (Dec 2020). https://doi.org/10.1007/978-3-030-64837-4_15
29. Hu, K., Yap, T.: Perfect monomial prediction for modular addition. IACR Transactions on Symmetric Cryptology **2024**(3), 177–199 (Sep 2024). <https://doi.org/10.46586/tosc.v2024.i3.177-199>
30. Langlois, A., Stehlé, D.: Worst-case to average-case reductions for module lattices. DCC **75**(3), 565–599 (2015). <https://doi.org/10.1007/s10623-014-9938-4>
31. Lidl, R., Niederreiter, H.: Finite Fields. Encyclopedia of Mathematics and its Applications, Cambridge University Press, 2 edn. (1996). <https://doi.org/10.1017/CB09780511525926>
32. Lyubashevsky, V., Ducas, L., Kiltz, E., Lepoint, T., Schwabe, P., Seiler, G., Stehlé, D., Bai, S.: CRYSTALS-Dilithium Algorithm Specifications and Supporting Documentation. NIST Post-Quantum Cryptography Standard (2022)
33. Lyubashevsky, V., Peikert, C., Regev, O.: On ideal lattices and learning with errors over rings. In: Gilbert, H. (ed.) EUROCRYPT 2010. LNCS, vol. 6110, pp. 1–23 (May / Jun 2010). https://doi.org/10.1007/978-3-642-13190-5_1
34. Mouha, N., Velichkov, V., De Cannière, C., Preneel, B.: The differential analysis of S-functions. In: Biryukov, A., Gong, G., Stinson, D.R. (eds.) SAC 2010. LNCS, vol. 6544, pp. 36–56 (Aug 2011). https://doi.org/10.1007/978-3-642-19574-7_3
35. Regev, O.: On lattices, learning with errors, random linear codes, and cryptography. In: Gabow, H.N., Fagin, R. (eds.) 37th ACM STOC. pp. 84–93. ACM Press (May 2005). <https://doi.org/10.1145/1060590.1060603>
36. Schönhage, A., Strassen, V.: Schnelle multiplikation großer zahlen. Computing **7**, 281–292 (1971). <https://doi.org/10.1007/BF02242355>
37. Schwabe, P., Avanzi, R., Bos, J., Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schanck, J.M., Seiler, G., Stehlé, D., Ding, J.: CRYSTALS-Kyber Algorithm Specifications and Supporting Documentation. NIST Post-Quantum Cryptography Standard (2022)
38. Steiner, M.J.: The complexity of algebraic algorithms for LWE. In: Joye, M., Leander, G. (eds.) EUROCRYPT 2024, Part III. LNCS, vol. 14653, pp. 375–403 (May 2024). https://doi.org/10.1007/978-3-031-58734-4_13
39. Sun, C., Tibouchi, M., Abe, M.: Revisiting the hardness of binary error LWE. In: ACISP. Lecture Notes in Computer Science, vol. 12248, pp. 425–444. Springer (2020)
40. Todo, Y.: Structural evaluation by generalized integral property. In: Oswald, E., Fischlin, M. (eds.) EUROCRYPT 2015, Part I. LNCS, vol. 9056, pp. 287–314 (Apr 2015). https://doi.org/10.1007/978-3-662-46800-5_12
41. Wiedemann, D.: Solving sparse linear equations over finite fields. IEEE transactions on information theory **32**(1), 54–62 (1986)
42. Zhang, Y., Lu, X., Liu, Y., Yin, Y., Wang, K.: LEAP: high-performance lattice-based pseudorandom number generator. IACR Trans. Symmetric Cryptol. **2025**(3), 151–182 (2025)

A Implementation considerations

A.1 Implementation details of Algorithms 1 and 2

This section is dedicated to details about our implementation of the subroutines of Algorithms 1 and 2.

Computing the ANF of $H_{\mathbf{c}}$. Let $h_c(a, x) := (a \times x)^c$ for any $0 \leq c \leq m-1$. For any $\mathbf{c}$, since $H_{\mathbf{c}} = \prod_{i=0}^{n-1} (a_i \times x_i)^{c_i}$, it holds that:

$$\text{Exp}(H_{\mathbf{c}}) = \text{Exp}(h_{c_0}) \times \dots \times \text{Exp}(h_{c_{m-1}})$$

since the variables x_i and a_i are all independent. Further, we emphasize that the ANF of the functions $h_c(a, x) = (a \times x)^c$ for any $0 \leq c \leq m$ are pre-generated and stored rather than re-computed every time Algorithm 1 is ran.

Computing the value of $|\text{Stab}(\mathbf{c})|$. For any $\mathbf{c} \in \mathbb{N}^n$, we denote by $S(\mathbf{c})$ the set of coefficients of $\mathbf{c}$, that is, $S(\mathbf{c}) = \{c_i\}_{0 \leq i \leq n-1}$. For any $c \in S(\mathbf{c})$, let $\mathcal{I}_c(\mathbf{c}) = \{i \in \llbracket 0, n-1 \rrbracket \mid c_i = c\}$. With the introduced notation, the cardinality of $\text{Stab}(\mathbf{c})$ can be efficiently computed, since it holds that:

$$|\text{Stab}(\mathbf{c})| = \sum_{c \in S(\mathbf{c})} |\mathcal{I}_c(\mathbf{c})|! .$$

Indeed, $\sigma \in \mathfrak{S}_n$ stabilizes $\mathbf{c}$ if and only if it permutes coordinates that are equal.

Computing the value of $|\text{Orb}(\mathbf{z}, \mathbf{t})|$. This step must be performed efficiently, because computing this orbit naively (*i.e.* by trying all possible permutations) costs $n!$. Since $|\text{Orb}(\mathbf{z}, \mathbf{t})| = n! / |\text{Stab}(\mathbf{z}, \mathbf{t})|$, it is sufficient to compute $|\text{Stab}(\mathbf{z}, \mathbf{t})|$. So, similarly to the previous computation, for any $(\mathbf{u}, \mathbf{v}) \in \mathbb{N}^n \times \mathbb{N}^n$, we denote by $S(\mathbf{u}, \mathbf{v}) := \{(u_i, v_i)\}_{0 \leq i \leq n-1}$ and for any $(u, v) \in S(\mathbf{u}, \mathbf{v})$, we let $\mathcal{I}_{(u,v)}(\mathbf{u}, \mathbf{v})$ be the set $\mathcal{I}_{(u,v)}(\mathbf{u}, \mathbf{v}) = \{i \in \llbracket 0, n-1 \rrbracket \mid u_i = u, v_i = v\}$. With a similar reasoning, we obtain:

$$|\text{Stab}(\mathbf{z}, \mathbf{t})| = \sum_{(u,v) \in S(\mathbf{z}, \mathbf{t})} |\mathcal{I}_{u,v}(\mathbf{z}, \mathbf{t})|! .$$

Computing the representative $(\mathbf{u}, \mathbf{v})^$ of $\text{Orb}(\mathbf{u}, \mathbf{v})$.* The output of Algorithm 1 depends on the system of representatives $\mathcal{D}^n$, but would work with any other system. We choose a system which enables us simple and fast computations. Our representative $(\mathbf{u}, \mathbf{v})^*$ is the smallest element of $\text{Orb}(\mathbf{u}, \mathbf{v})$ for the lexicographic order over $\mathbb{N}^2$. Stated otherwise, for any $(\mathbf{u}, \mathbf{v})$, the representative $(\mathbf{u}, \mathbf{v})^*$ is computed by sorting all pairs (u_i, v_i) using the following order over $\mathbb{N}^2$:

$$\forall a, b, c, d \in \mathbb{N}, \quad (a, b) <_{\text{lex}} (c, d) \iff b < d \text{ or } (b = d \text{ and } a < c). \quad (12)$$

The fact that we first compare the second coordinates and then only the first ones is on purpose. The relevancy of this choice is clear with Algorithm 3.

Choice of $\mathcal{C}_m^n$. In practice, for vectors of $\mathbb{N}^n$, we also choose the minimal vector of each orbit for the lexicographical order as the system of representatives $\mathcal{C}_m^n$.

A.2 Implementation details of Algorithm 3

As with the first two algorithms, we aim at an efficient implementation of Algorithm 3. We continue using $\mathcal{D}^n$ made of the smallest vectors of each class for the order defined in Equation 12. Furthermore, we also choose $\mathcal{C}^n$ so that $\mathbf{v}^*$ can be computed by sorting the coordinates of $\mathbf{v}$ in increasing order. This way, for any $(\mathbf{u}, \mathbf{v}) \in \text{Exp}(F^{m,n})$, τ can be chosen to be the identity since it holds that the second coordinate of $(\mathbf{u}, \mathbf{v})^*$ is necessarily $\mathbf{v}^*$.

Computing the set $\{\mathbf{a}^{\sigma \cdot \tau \cdot \mathbf{u}}, \sigma \in \text{Stab}(\mathbf{v}^)\}$.* Algorithm 3 therefore sums up to the computation of the sets $\{\mathbf{a}^{\sigma \cdot \mathbf{u}}, \sigma \in \text{Stab}(\mathbf{v})\}$ for all $(\mathbf{u}, \mathbf{v}) \in \text{Exp}(F^{m,n})$ since $\mathbf{v}^* = \mathbf{v}$. As we are looking for all permutations σ such that $\sigma \cdot \mathbf{v} = \mathbf{v}$, we can independently look at the subvectors of $\mathbf{u}$, namely $\mathbf{z}_v := (u_i)_{i \in \mathcal{I}_v(\mathbf{v})}$ for all $v \in S(\mathbf{v})$. Up to the proper reordering of the coordinates, the set $\{\mathbf{a}^{\sigma \cdot \mathbf{u}}, \sigma \in \text{Stab}(\mathbf{v})\}$ can be expressed as:

$$\{\mathbf{a}^{\sigma \cdot \mathbf{u}}, \sigma \in \text{Stab}(\mathbf{v})\} \simeq \bigtimes_{v \in S(\mathbf{v})} \text{Orb}(\mathbf{w}_v).$$

All in all, Algorithm 3 sums up to enumerating the orbit of these subvectors. This corresponds to enumerate the permutations of a multiset.

B Pseudo-code of Algorithm 4

Algorithm 4 Computing a system of representatives of $Q^{m,n} / \sim$.

```

1: Initialize an empty dictionary Sys_of_reps.
2: for all  $(v^*, \alpha_{v^*}) \in \text{Pairs}_x(F^{m,n})$  do ▷ Algorithm 3
3:   Initialise the entry Sys_of_reps $[\alpha_{v^*}]$ .
4:   IsNewQ = True,  $s = |\text{Supp}(\alpha_{v^*})|$ 
5:   for all  $\sigma \in \mathfrak{S}_s / \text{Stab}(v^*)$  do
6:      $\beta \leftarrow \sigma \cdot \alpha_{v^*}$ 
7:     if Sys_of_reps $[\beta] \neq \emptyset$  then
8:       Append  $\sigma \cdot v^*$  to Sys_of_reps $[\beta]$ 
9:       if  $\beta \neq \alpha_{v^*}$  then
10:        IsNewQ = False
11:      end if
12:    end if
13:  end for
14:  if  $\neg \text{IsNewQ}$  then
15:    Delete the entry Sys_of_reps $[\alpha_{v^*}]$ 
16:  end if
17: end for
18: return Sys_of_reps.

```

C Graphic representation of the ANF of $F^{4,4}$

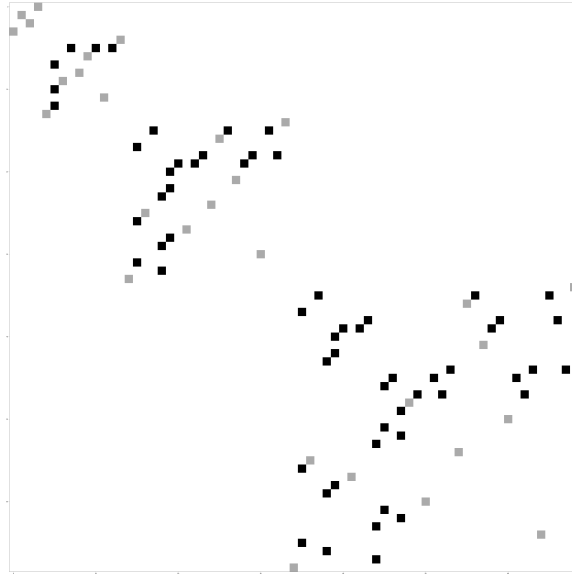


Fig. 3: ANF of $F^{4,4}$ where the terms $a_{3,i}x_{3,i}$, for $0 \leq i \leq 3$ are omitted. A dark square at coordinate $(\mathbf{u}, \mathbf{v})$ indicates that $\mathbf{a}^{\mathbf{u}}\mathbf{x}^{\mathbf{v}}$ appears, a given row thus provides the ANF of some $\alpha_{\mathbf{v}}$. The row is darker if $\alpha_{\mathbf{v}}$ contains more terms. For each column containing 3 terms, the corresponding rows are all equal, thus indicating a rank defect.